CHAROTAR UNIVERSITY OF SCIENCE AND TECHNOLOGY

Chandubhai S. Patel Institute of Technology (CSPIT)

K.D. Patel Department of Information Technology • Academic Year 2026–2027

Name : Rydham Desai	**ID :** 25IT019	**Branch :** IT
DEPT : CSPIT	**SUBJECT :** OOP (CEUE203)	**Semester :** 3rd Semester

INDEX OF PRACTICAL IMPLEMENTATIONS

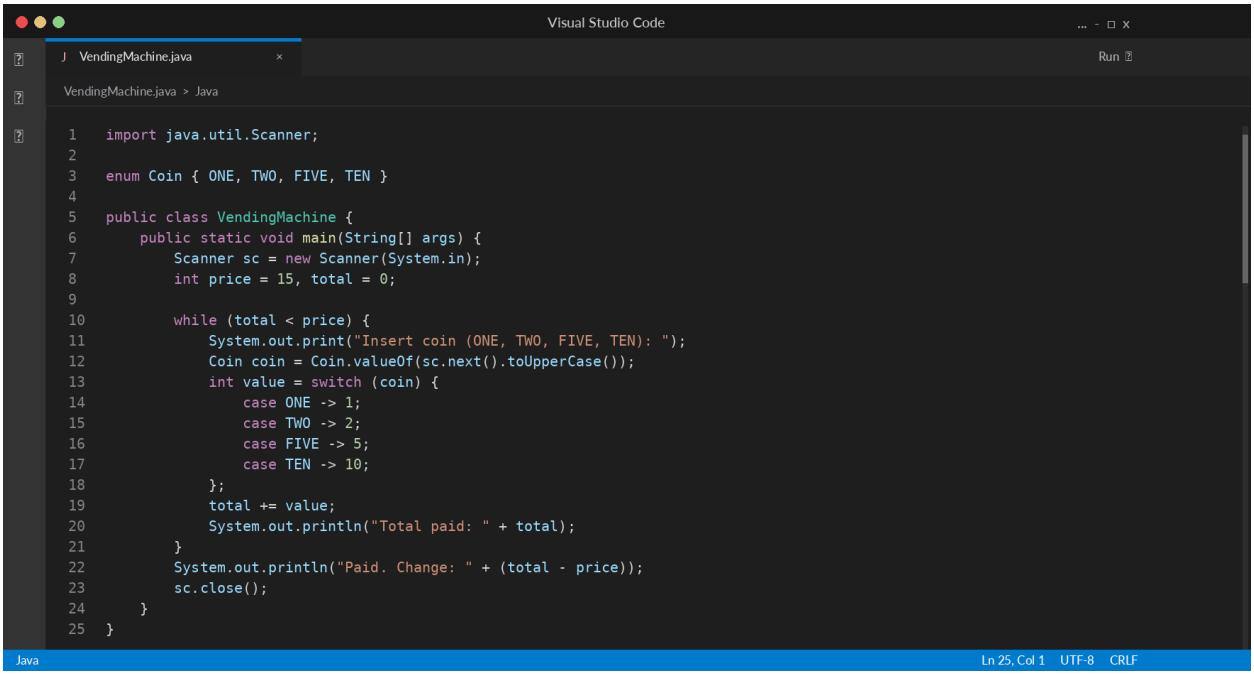
Prac	Title / Practical Problem Statement	Page No.	Date	Sign
01	Vending Machine: Coin enums & switch expressions Toll Booth: Vehicle records & frequency counter RPSLS: Five-move game (Advanced Learners)	**2 – 7**	07/07/2026	
02	Smart Thermostat: Encapsulation & constructor chaining Cinema Show: Transactional entity & state tracking Parking Lot: Capacity and occupancy (Advanced Learners)	**8 – 13**	13/07/2026	
03	Distinct Points: equals() & hashCode() Duplicate Card Check: Composite equality & search Fractions: Fraction reduction & comparison (Advanced Learners)	**14 – 19**	20/07/2026	
04	Password Strength: Regex-based security validation Chat-Log Filter: Case-insensitive filtering & StringBuilder Template Filler: Regex placeholders (Advanced Learners)	**20 – 25**	29/07/2026	
05	Shape Areas: Abstract classes & dynamic method dispatch Payroll System: Superclass delegation & subclass polymorphism Media Late Fees: Abstract media hierarchy (Advanced Learners)	**26 – 31**	03/08/2026	
06	Remote Control: Interfaces, default methods & lambda predicates Notification Senders: Functional interfaces & urgent markers Discount Engine: Lambda-based discount rules (Advanced Learners)	**32 – 37**	17/08/2026	
07	Form Validator: Declarative annotations & reflection Mini Test Runner: Dynamic method discovery & @Run execution Column Mapper: Runtime annotation field mapping (Advanced Learners)	**38 – 43**	27/10/2026	
08	Guarded Calculator: DivideByZeroException & multi-catch recovery Stock Issue: Contextual OutOfStockException & inventory handling AutoCloseable Resource: Resource lifecycle (Advanced Learners)	**44 – 49**	16/09/2026	
09	Counter Race: Thread synchronization & race-condition mitigation Seat Booking Race: Monitor locking against ticket overselling Parallel Array Sum: Shared-total race and fixes (Advanced Learners)	**50 – 55**	21/09/2026	
10	Thread Pool Runner: ExecutorService worker reuse & lifecycle Producer–Consumer: Guarded wait()/notify() coordination Deadlock Demonstration: Opposite locks & consistent lock order (Advanced Learners)	**56 – 61**	05/10/2026	
11	Save/Load Serialization: Binary persistence & transient fields Log Analyzer: NIO.2 stream pipelines & file attributes NIO Directory Tree Report: File metadata report (Advanced Learners)	**62 – 67**	12/10/2026	
12	Word Frequency: HashMap counting & Comparator sorting Unique Visitors: Data deduplication via HashSet & LinkedHashSet Leaderboard: Map-based top-N ranking (Advanced Learners)	**68 – 73**	19/10/2026	
13	Generic Containers: Box<T>, Pair<A,B> & generic methods Bounded Stats: Upper-bounded type parameter <N extends Number> Generic Stack: Array-backed generic container (Advanced Learners)	**74 – 79**	26/10/2026	
14	Assemble and Test: Layered application & unit tests Modular Packaging: Packages, runnable JAR & persistence test (Advanced Learners)	**80 – 83**	27/10/2026	

Total Practicals Completed: 14 / 14 **Faculty In-Charge Signature:** _____ **Date:** _____

PRACTICAL - 01

Aim: Vending machine (VendingMachine.java)

Code Screenshot:



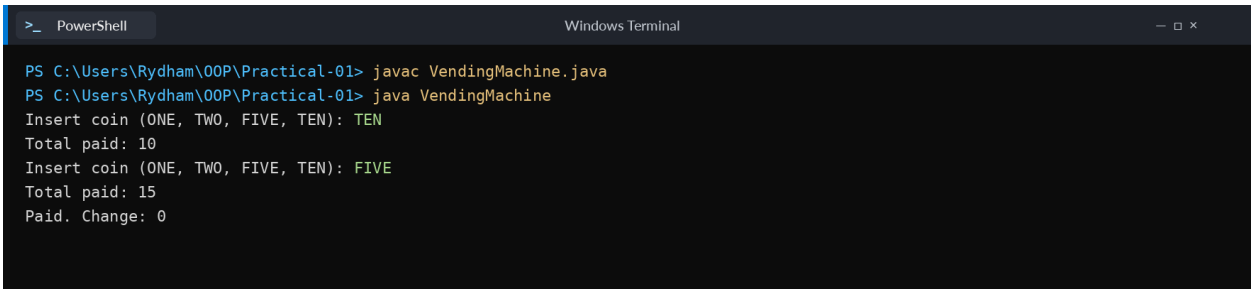

```java
import java.util.Scanner;

enum Coin { ONE, TWO, FIVE, TEN }

public class VendingMachine {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int price = 15, total = 0;

        while (total < price) {
            System.out.print("Insert coin (ONE, TWO, FIVE, TEN): ");
            Coin coin = Coin.valueOf(sc.next().toUpperCase());
            int value = switch (coin) {
                case ONE -> 1;
                case TWO -> 2;
                case FIVE -> 5;
                case TEN -> 10;
            };
            total += value;
            System.out.println("Total paid: " + total);
        }
        System.out.println("Paid. Change: " + (total - price));
        sc.close();
    }
}
```

Output Screenshot:

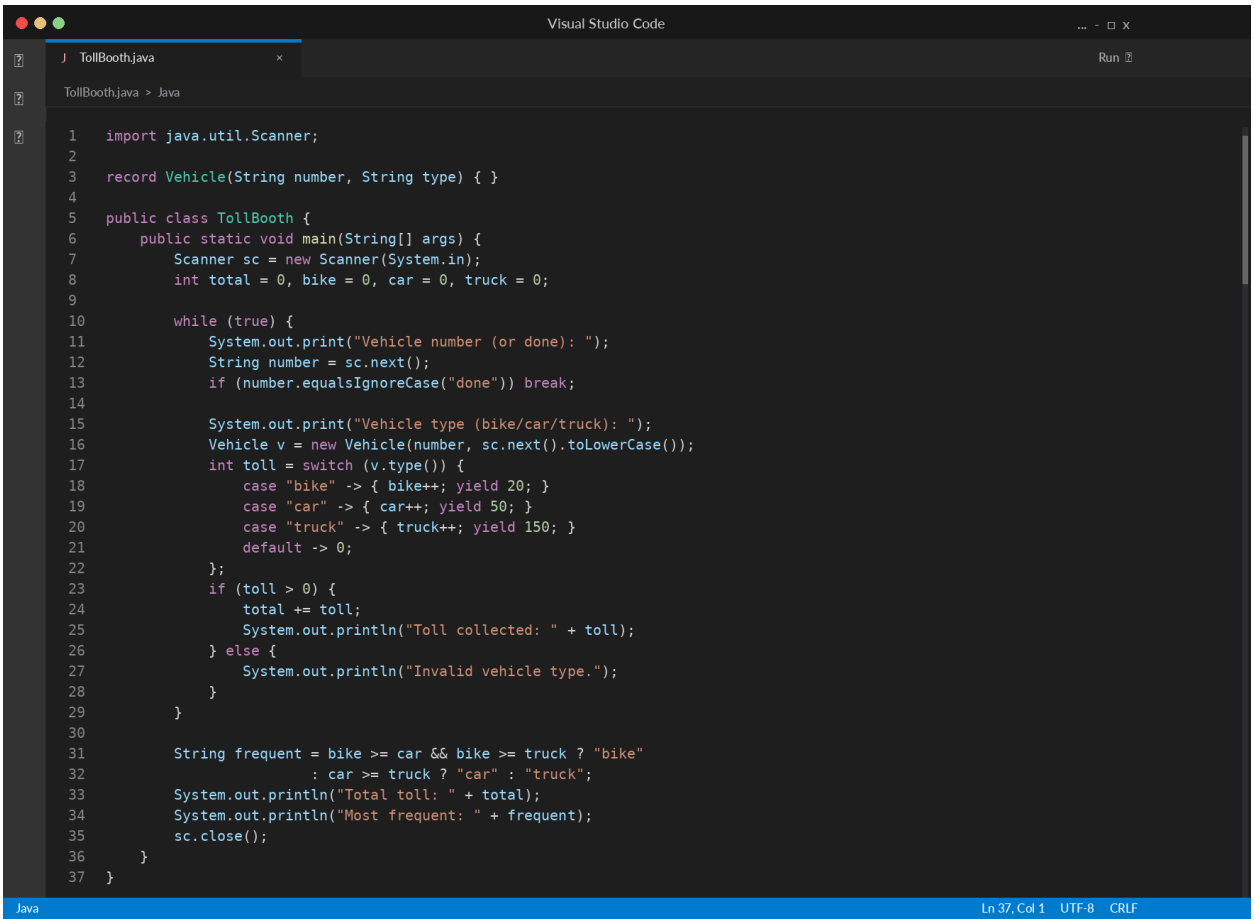


Learning Outcome:

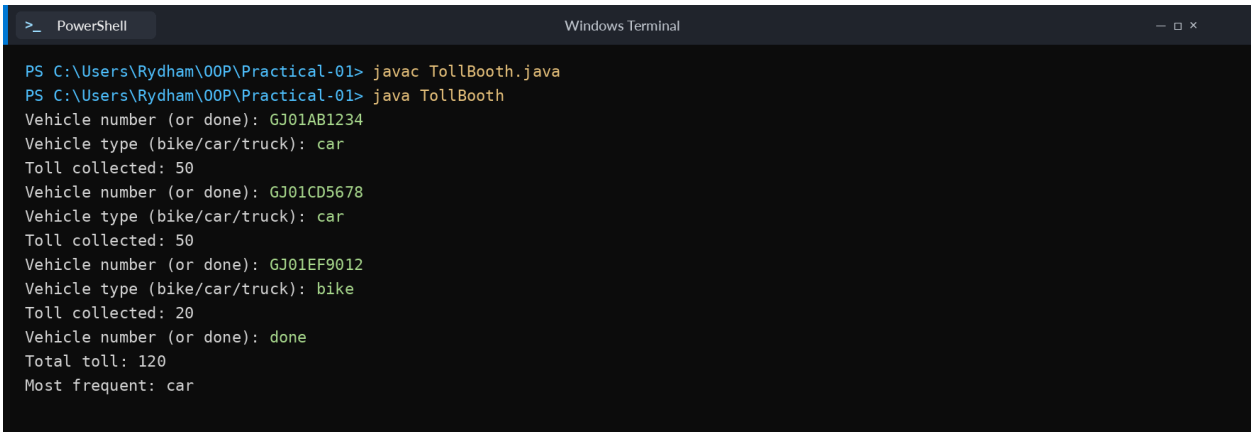
- Learned how to define and utilize Java enum types for fixed sets of constants.
- Understood modern switch expressions with arrow syntax and return values.
- Implemented accumulator logic for iterative coin transaction management.
- Handled console input parsing and enum constant resolution via valueOf().
- Computed exact monetary change based on user input.

Aim: Toll booth (TollBooth.java)

Code Screenshot:



Output Screenshot:

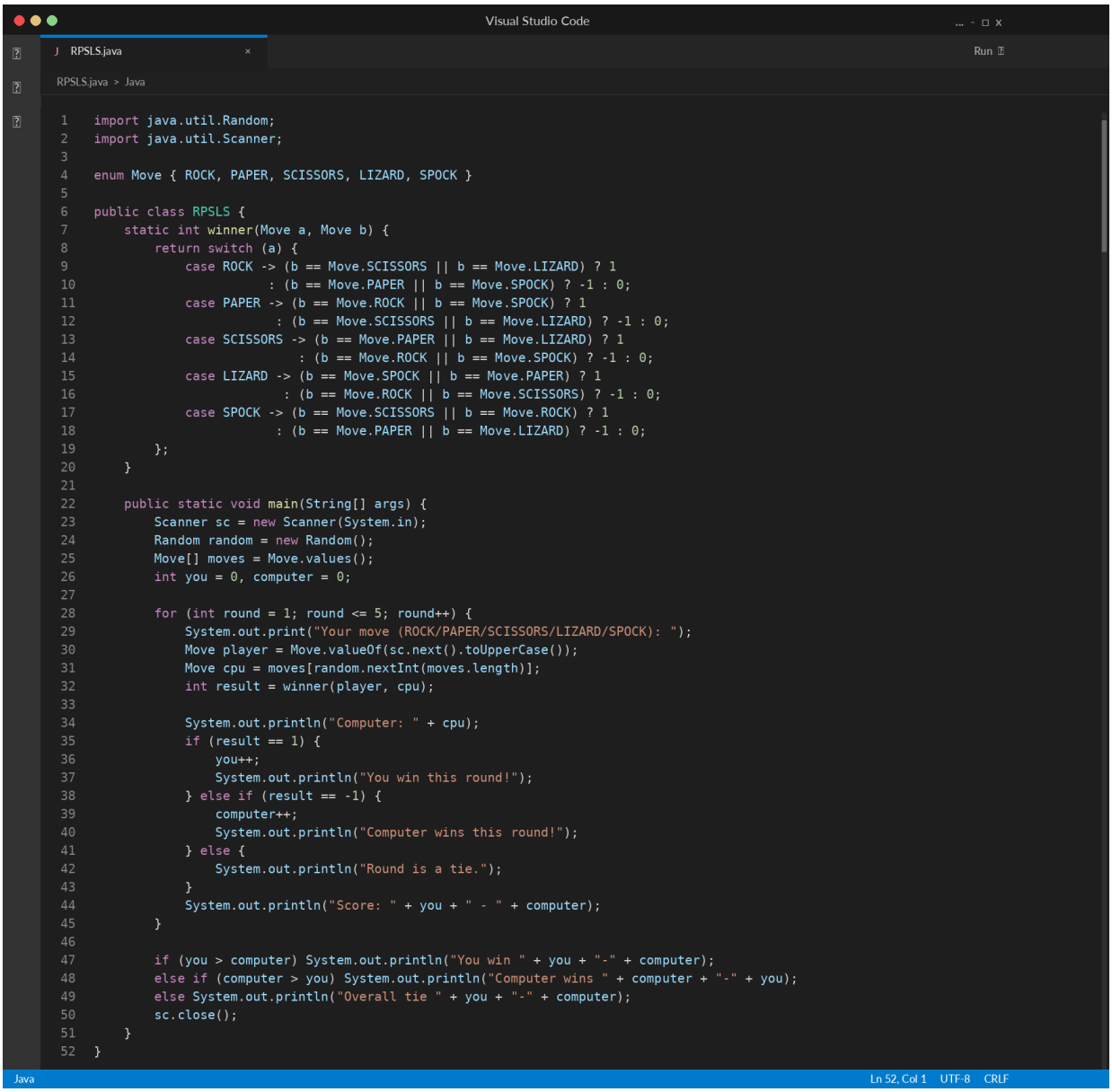


Learning Outcome:

- Mastered Java record types for concise, immutable data holding.
- Used switch expressions with multi-statement blocks and the yield statement.
- Implemented frequency tracking counters across heterogeneous categories.
- Applied loop control flows with sentinel termination conditions.
- Gained hands-on experience in structured object-oriented problem solving.

Aim: Rock-Paper-Scissors-Lizard-Spock (RPSLS.java)

Code Screenshot:




```java
import java.util.Random;
import java.util.Scanner;

enum Move { ROCK, PAPER, SCISSORS, LIZARD, SPOCK }

public class RPSLS {
    static int winner(Move a, Move b) {
        return switch (a) {
            case ROCK -> (b == Move.SCISSORS || b == Move.LIZARD) ? 1
                       : (b == Move.PAPER || b == Move.SPOCK) ? -1 : 0;
            case PAPER -> (b == Move.ROCK || b == Move.SPOCK) ? 1
                        : (b == Move.SCISSORS || b == Move.LIZARD) ? -1 : 0;
            case SCISSORS -> (b == Move.PAPER || b == Move.LIZARD) ? 1
                           : (b == Move.ROCK || b == Move.SPOCK) ? -1 : 0;
            case LIZARD -> (b == Move.SPOCK || b == Move.PAPER) ? 1
                         : (b == Move.ROCK || b == Move.SCISSORS) ? -1 : 0;
            case SPOCK -> (b == Move.SCISSORS || b == Move.ROCK) ? 1
                        : (b == Move.PAPER || b == Move.LIZARD) ? -1 : 0;
        };
    }

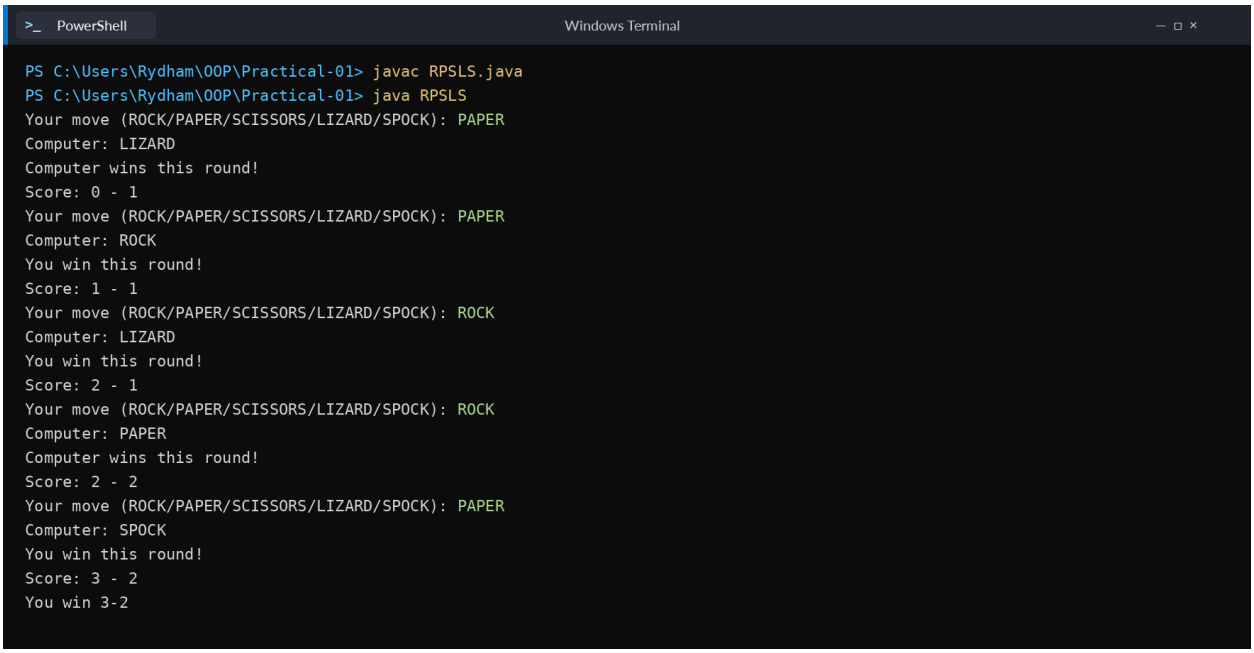
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Random random = new Random();
        Move[] moves = Move.values();
        int you = 0, computer = 0;

        for (int round = 1; round <= 5; round++) {
            System.out.print("Your move (ROCK/PAPER/SCISSORS/LIZARD/SPOCK): ");
            Move player = Move.valueOf(sc.next().toUpperCase());
            Move cpu = moves[random.nextInt(moves.length)];
            int result = winner(player, cpu);

            System.out.println("Computer: " + cpu);
            if (result == 1) {
                you++;
                System.out.println("You win this round!");
            } else if (result == -1) {
                computer++;
                System.out.println("Computer wins this round!");
            } else {
                System.out.println("Round is a tie.");
            }
            System.out.println("Score: " + you + " - " + computer);
        }

        if (you > computer) System.out.println("You win " + you + "-" + computer);
        else if (computer > you) System.out.println("Computer wins " + computer + "-" + you);
        else System.out.println("Overall tie " + you + "-" + computer);
        sc.close();
    }
}
```

Output Screenshot:



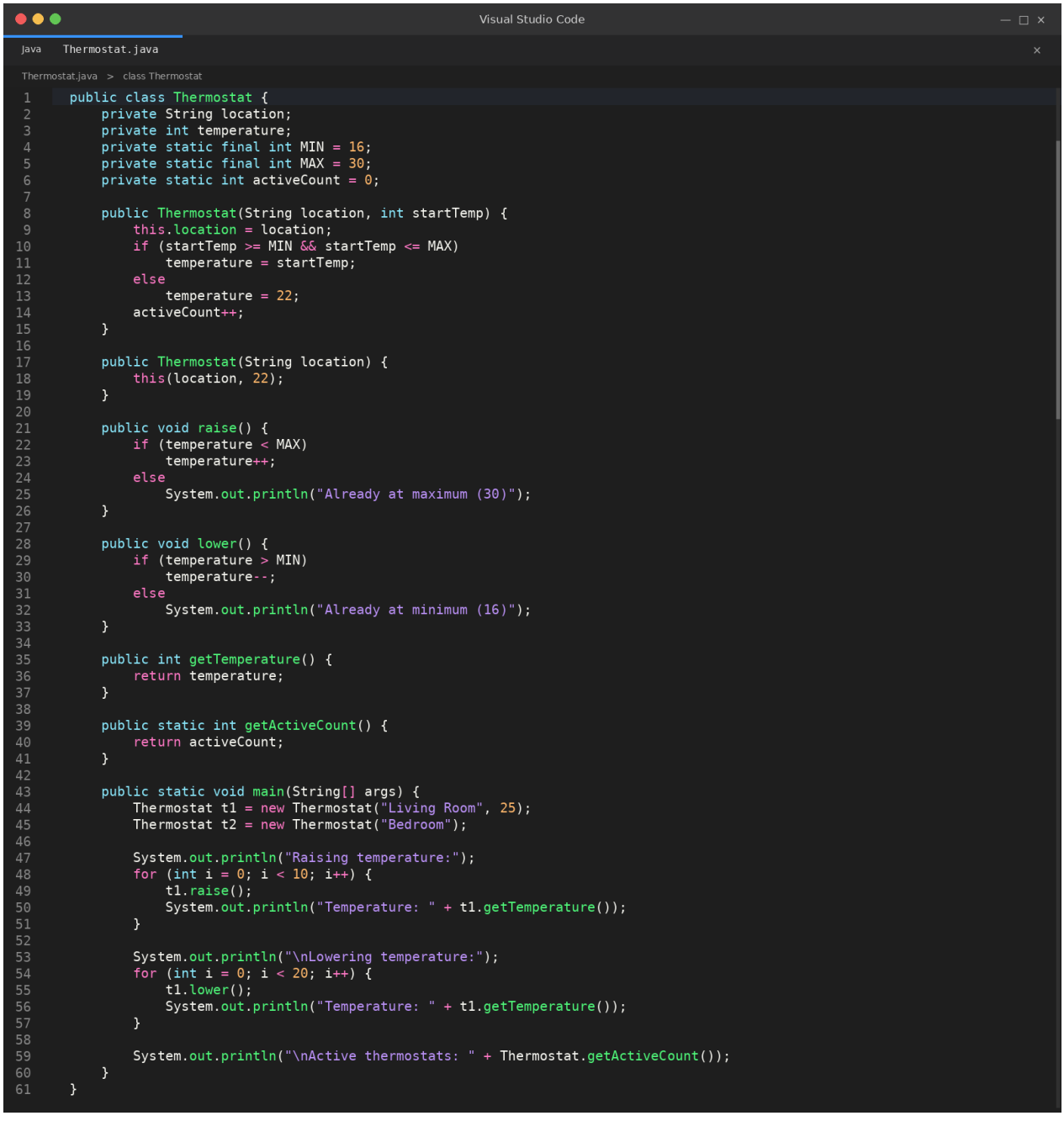
Learning Outcome:

- Defined Java enum constants to represent the five game moves.
- Used a switch expression to determine the winner for each pair of moves.
- Implemented randomized computer selections and five-round gameplay.
- Maintained separate scores for the player and computer, including ties.
- Applied conditional logic to display the overall match winner.

PRACTICAL - 02

Aim: Smart thermostat (Thermostat.java)

Code Screenshot:




```java
public class Thermostat {
    private String location;
    private int temperature;
    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;
        if (startTemp >= MIN && startTemp <= MAX)
            temperature = startTemp;
        else
            temperature = 22;
        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX)
            temperature++;
        else
            System.out.println("Already at maximum (30)");
    }

    public void lower() {
        if (temperature > MIN)
            temperature--;
        else
            System.out.println("Already at minimum (16)");
    }

    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

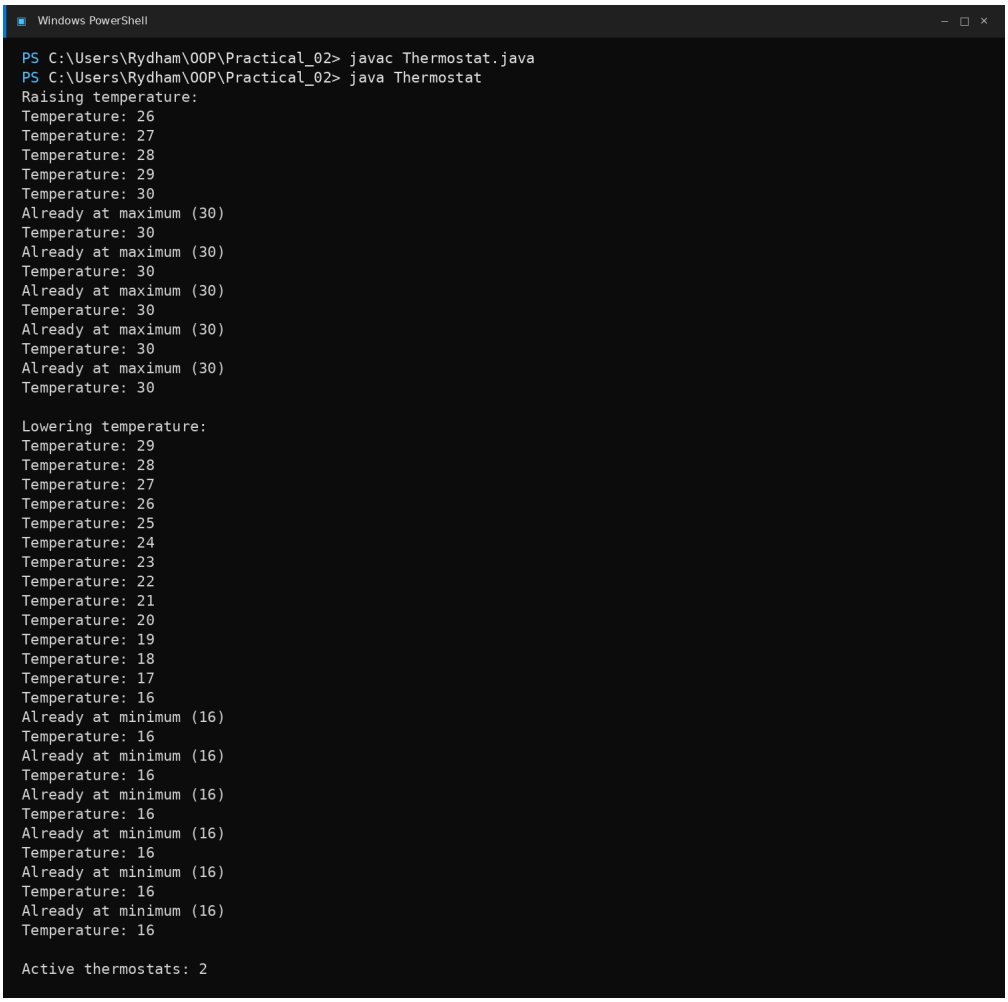
    public static void main(String[] args) {
        Thermostat t1 = new Thermostat("Living Room", 25);
        Thermostat t2 = new Thermostat("Bedroom");

        System.out.println("Raising temperature:");
        for (int i = 0; i < 10; i++) {
            t1.raise();
            System.out.println("Temperature: " + t1.getTemperature());
        }

        System.out.println("\nLowering temperature:");
        for (int i = 0; i < 20; i++) {
            t1.lower();
            System.out.println("Temperature: " + t1.getTemperature());
        }

        System.out.println("\nActive thermostats: " + Thermostat.getActiveCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned encapsulation principles by restricting direct field access with private.
- Implemented constructor overloading and constructor chaining using this(...).
- Managed shared class state across instances using static variables.
- Enforced boundary constraints within mutator methods.
- Understood how objects maintain independent internal state.

Aim: Cinema show (CinemaShow.java)

Code Screenshot:

Visual Studio Code

Java CinemaShow.java

CinemaShow.java > class CinemaShow

```java
public class CinemaShow {
    private String title;
    private int seatsAvailable;
    private final int capacity;
    private static int totalBooked = 0;

    public CinemaShow(String title, int capacity) {
        this.title = title;
        this.capacity = capacity;
        seatsAvailable = capacity;
    }

    public CinemaShow(String title) {
        this(title, 100);
    }

    public boolean book(int n) {
        if (n > 0 && n <= seatsAvailable) {
            seatsAvailable -= n;
            totalBooked += n;
            return true;
        }
        return false;
    }

    public void cancel(int n) {
        if (n > 0) {
            seatsAvailable += n;
            if (seatsAvailable > capacity)
                seatsAvailable = capacity;
        }
    }

    public int getSeatsAvailable() {
        return seatsAvailable;
    }

    public static int getTotalBooked() {
        return totalBooked;
    }

    public static void main(String[] args) {
        CinemaShow show = new CinemaShow("Evening Show", 5);
        CinemaShow matinee = new CinemaShow("Matinee");

        System.out.println("Matinee seats: " + matinee.getSeatsAvailable());

        System.out.println("Book 3: " + show.book(3));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 2: " + show.book(2));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 1: " + show.book(1));
        System.out.println("Seats available: " + show.getSeatsAvailable());

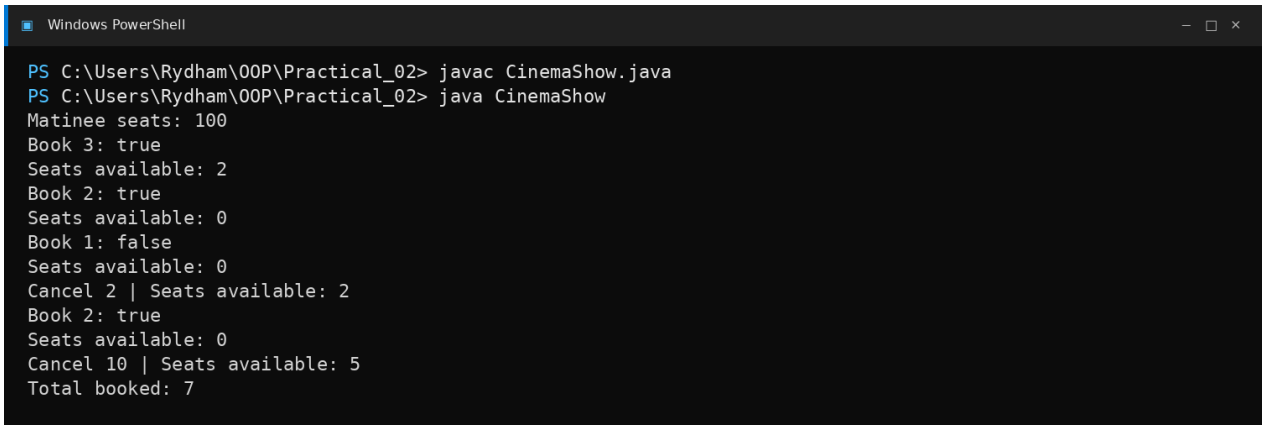
        show.cancel(2);
        System.out.println("Cancel 2 | Seats available: " + show.getSeatsAvailable());

        System.out.println("Book 2: " + show.book(2));
        System.out.println("Seats available: " + show.getSeatsAvailable());

        show.cancel(10);
        System.out.println("Cancel 10 | Seats available: " + show.getSeatsAvailable());

        System.out.println("Total booked: " + CinemaShow.getTotalBooked());
    }
}
```

Output Screenshot:

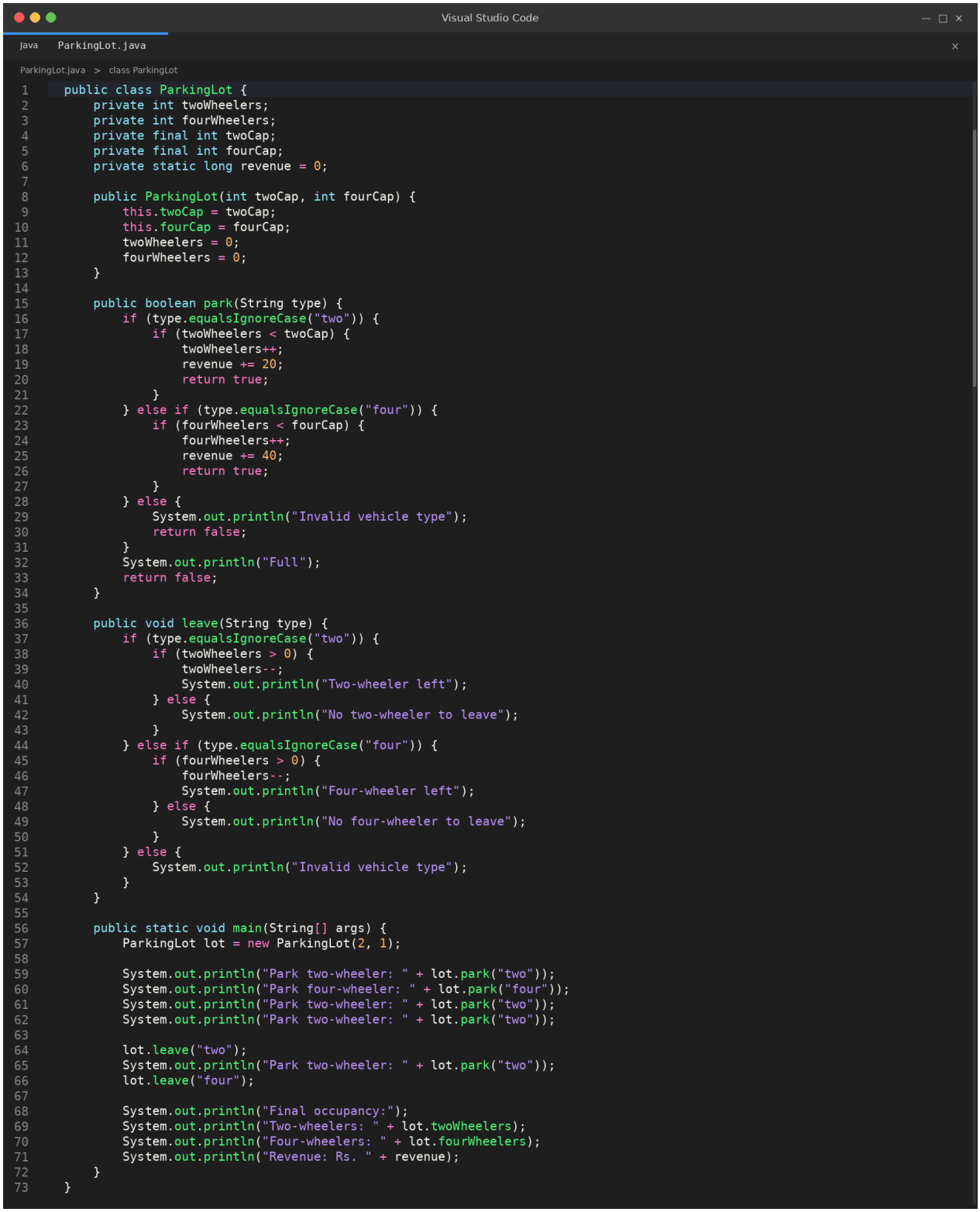


Learning Outcome:

- Mastered modeling real-world transactional entities in Java.
- Implemented defensive programming logic to protect capacity bounds.
- Tracked global operational metrics with static member variables.
- Applied boolean response status for state-modifying actions.
- Practiced clean method signatures and separation of concerns.

Aim: Parking lot (ParkingLot.java) - Advanced Learners

Code Screenshot:




```java
public class ParkingLot {
    private int twoWheelers;
    private int fourWheelers;
    private final int twoCap;
    private final int fourCap;
    private static long revenue = 0;

    public ParkingLot(int twoCap, int fourCap) {
        this.twoCap = twoCap;
        this.fourCap = fourCap;
        twoWheelers = 0;
        fourWheelers = 0;
    }

    public boolean park(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers < twoCap) {
                twoWheelers++;
                revenue += 20;
                return true;
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers < fourCap) {
                fourWheelers++;
                revenue += 40;
                return true;
            }
        } else {
            System.out.println("Invalid vehicle type");
            return false;
        }
        System.out.println("Full");
        return false;
    }

    public void leave(String type) {
        if (type.equalsIgnoreCase("two")) {
            if (twoWheelers > 0) {
                twoWheelers--;
                System.out.println("Two-wheeler left");
            } else {
                System.out.println("No two-wheeler to leave");
            }
        } else if (type.equalsIgnoreCase("four")) {
            if (fourWheelers > 0) {
                fourWheelers--;
                System.out.println("Four-wheeler left");
            } else {
                System.out.println("No four-wheeler to leave");
            }
        } else {
            System.out.println("Invalid vehicle type");
        }
    }

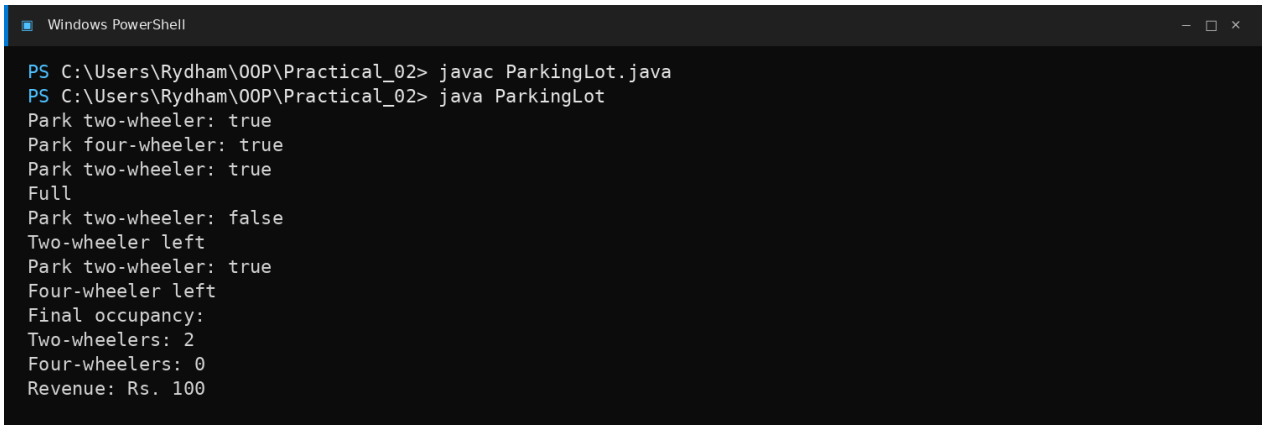
    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 1);

        System.out.println("Park two-wheeler: " + lot.park("two"));
        System.out.println("Park four-wheeler: " + lot.park("four"));
        System.out.println("Park two-wheeler: " + lot.park("two"));
        System.out.println("Park two-wheeler: " + lot.park("two"));

        lot.leave("two");
        System.out.println("Park two-wheeler: " + lot.park("two"));
        lot.leave("four");

        System.out.println("Final occupancy:");
        System.out.println("Two-wheelers: " + lot.twoWheelers);
        System.out.println("Four-wheelers: " + lot.fourWheelers);
        System.out.println("Revenue: Rs. " + revenue);
    }
}
```

Output Screenshot:



Learning Outcome:

- Applied encapsulation to manage parking occupancy and capacity.
- Used final fields to enforce fixed parking-section limits.
- Implemented conditional logic to accept or reject vehicles.
- Updated static revenue based on successful parking events.
- Maintained valid occupancy when vehicles enter and leave.

PRACTICAL - 03

Aim: Distinct points (Point.java + Driver.java)

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Point.java

Point.java > ...

```java
import java.util.Objects;

public class Point {
    private int x, y;

    public Point(int x, int y) {
        this.x = x;
        this.y = y;
    }

    public String toString() {
        return "(" + x + ", " + y + ")";
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Point p)) return false;
        return x == p.x && y == p.y;
    }

    public int hashCode() {
        return Objects.hash(x, y);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

Driver.java

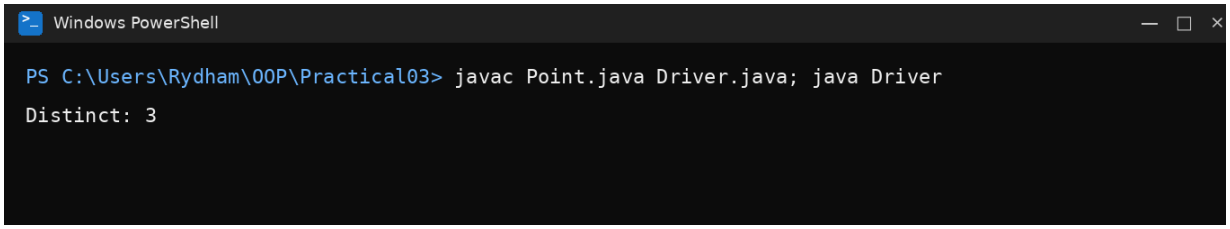
Driver.java > ...

```java
public class Driver {
    public static void main(String[] args) {
        Point[] points = {
            new Point(1, 2), new Point(3, 4),
            new Point(1, 2), new Point(5, 6),
            new Point(3, 4)
        };

        int distinct = 0;
        for (int i = 0; i < points.length; i++) {
            boolean found = false;
            for (int j = 0; j < i; j++) {
                if (points[i].equals(points[j])) {
                    found = true;
                    break;
                }
            }
            if (!found) distinct++;
        }
        System.out.println("Distinct: " + distinct);
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Mastered the contract between equals() and hashCode() in Java.
- Overrode Object.toString() for human-readable coordinate representation.
- Implemented array deduplication logic using equality checking.
- Learned pattern matching for instanceof introduced in modern Java.
- Understood hashing consistency requirements when overriding equals.

PRACTICAL - 03

Aim: Duplicate card check (Card.java + Driver.java)

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Card.java

Card.java > ...

```java
import java.util.Objects;

public class Card {
    private String rank, suit;

    public Card(String rank, String suit) {
        this.rank = rank;
        this.suit = suit;
    }

    public String toString() {
        return rank + " of " + suit;
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Card c)) return false;
        return Objects.equals(rank, c.rank)
            && Objects.equals(suit, c.suit);
    }

    public int hashCode() {
        return Objects.hash(rank, suit);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

Driver.java

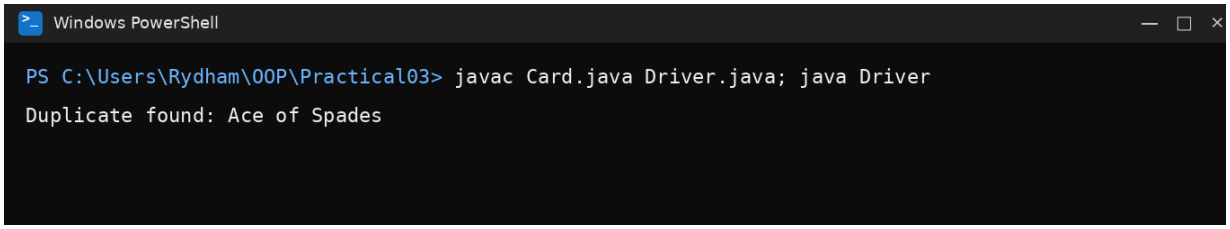
Driver.java > ...

```java
public class Driver {
    public static void main(String[] args) {
        Card[] input = {
            new Card("Ace", "Spades"),
            new Card("King", "Hearts"),
            new Card("Ace", "Spades")
        };
        Card[] cards = new Card[input.length];
        int count = 0;

        for (Card card : input) {
            boolean duplicate = false;
            for (int i = 0; i < count; i++) {
                if (card.equals(cards[i])) {
                    System.out.println("Duplicate found: " + card);
                    duplicate = true;
                    break;
                }
            }
            if (duplicate) break;
            cards[count++] = card;
        }
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Modeled compound domain entities with composite equality semantics.
- Applied string formatting in overridden toString() representations.
- Detected duplicates with early-exit search algorithms.
- Understood how Objects.equals() safely handles null references.
- Reinforced object identity vs structural equality concepts.

PRACTICAL - 03

Aim: Fractions (Fraction.java + Driver.java) - Advanced Learners

Code Screenshot:

File Edit Selection View Go Run Terminal Help

Fraction.java

```java
import java.util.Objects;

public class Fraction {
    private int num, den;

    public Fraction(int num, int den) {
        if (den == 0)
            throw new IllegalArgumentException("Denominator cannot be zero");
        int g = gcd(Math.abs(num), Math.abs(den));
        this.num = num / g;
        this.den = den / g;
        if (this.den < 0) {
            this.num = -this.num;
            this.den = -this.den;
        }
    }

    private int gcd(int a, int b) {
        while (b != 0) {
            int t = b;
            b = a % b;
            a = t;
        }
        return a;
    }

    public String toString() {
        return num + "/" + den;
    }

    public boolean equals(Object obj) {
        if (this == obj) return true;
        if (!(obj instanceof Fraction f)) return false;
        return num == f.num && den == f.den;
    }

    public int hashCode() {
        return Objects.hash(num, den);
    }
}
```

Java

File Edit Selection View Go Run Terminal Help

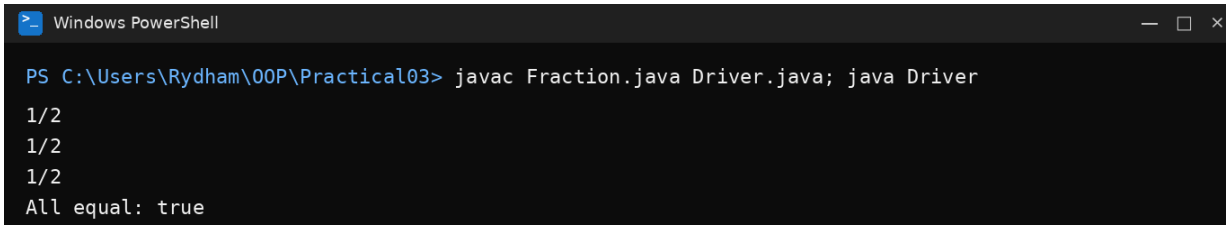
Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Fraction f1 = new Fraction(1, 2);
        Fraction f2 = new Fraction(2, 4);
        Fraction f3 = new Fraction(3, 6);

        System.out.println(f1);
        System.out.println(f2);
        System.out.println(f3);
        System.out.println("All equal: " +
            (f1.equals(f2) && f2.equals(f3)));
    }
}
```

Java

Output Screenshot:



Learning Outcome:

- Reduced fractions to their lowest terms using the GCD method.
- Overrode toString() to display fractions in readable form.
- Compared equivalent fractions using equals().
- Kept hashCode() consistent with the equality comparison.
- Represented rational values using a Java class.

PRACTICAL - 04

Aim: Password strength (PasswordChecker.java + Driver.java)

Code Screenshot:

PasswordChecker.java

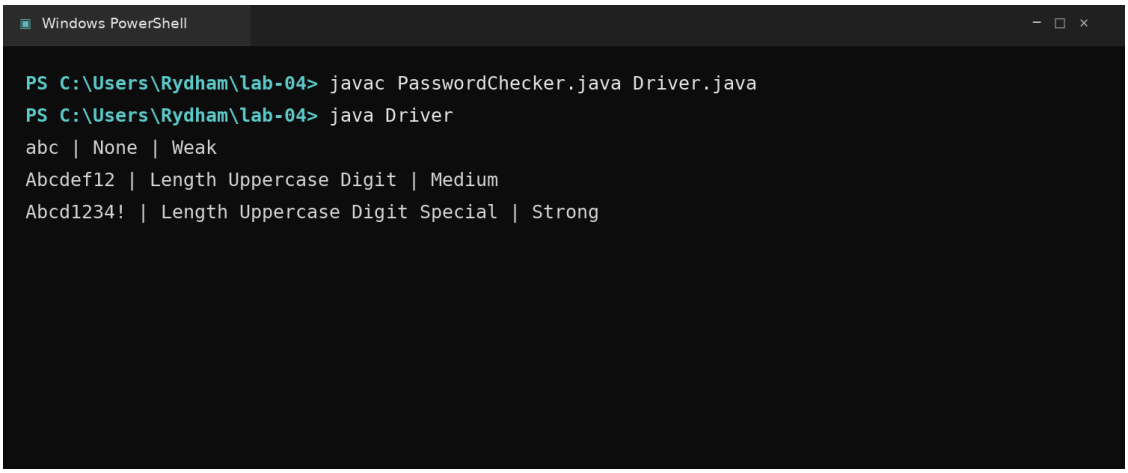
```java
public class PasswordChecker {
    public String strength(String pw) {
        int score = 0;
        if (pw.length() >= 8) score++;
        if (pw.matches(".*[A-Z].*")) score++;
        if (pw.matches(".*[0-9].*")) score++;
        if (pw.matches(".*[^A-Za-z0-9].*")) score++;
        if (score <= 1) return "Weak";
        if (score <= 3) return "Medium";
        return "Strong";
    }

    public void passedRules(String pw) {
        boolean found = false;
        if (pw.length() >= 8) { System.out.print("Length "); found = true; }
        if (pw.matches(".*[A-Z].*")) { System.out.print("Uppercase "); found = true; }
        if (pw.matches(".*[0-9].*")) { System.out.print("Digit "); found = true; }
        if (pw.matches(".*[^A-Za-z0-9].*")) { System.out.print("Special "); found = true; }
        if (!found) System.out.print("None");
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        PasswordChecker checker = new PasswordChecker();
        String[] passwords = {"abc", "Abcdef12", "Abcd1234!"};
        for (String pw : passwords) {
            System.out.print(pw + " | ");
            checker.passedRules(pw);
            System.out.println(" | " + checker.strength(pw));
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to validate passwords using multiple security rules.
- Understood the use of Regular Expressions (Regex) for pattern matching.
- Learned to use String methods such as length() and matches().
- Implemented password strength classification (Weak, Medium, Strong).
- Improved problem-solving and decision-making using conditional statements.

PRACTICAL - 04

Aim: Chat-log filter (ChatFilter.java + Driver.java)

Code Screenshot:

ChatFilter.java

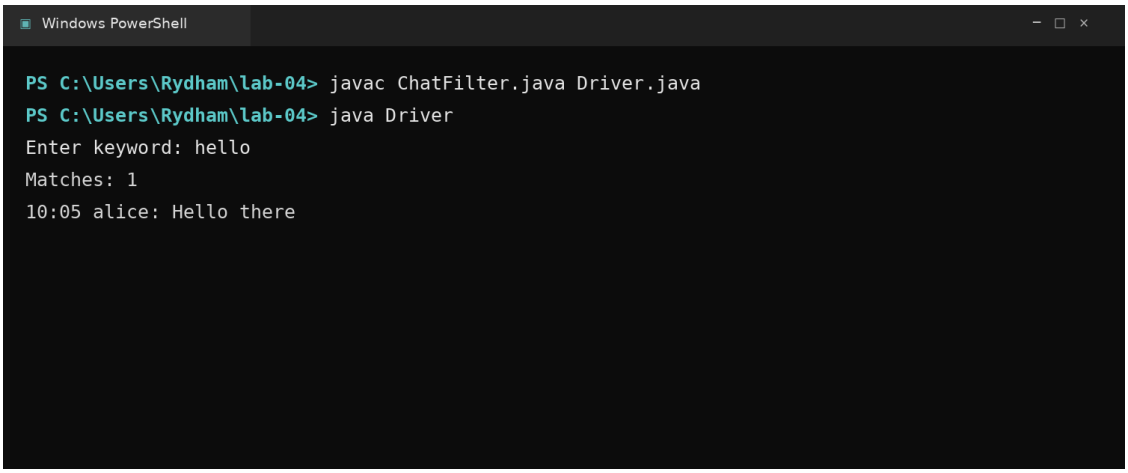
```java
public class ChatFilter {
    public String filter(String[] lines, String keyword) {
        StringBuilder report = new StringBuilder();
        int count = 0;
        for (String line : lines) {
            String[] parts = line.split(" ", 3);
            if (parts.length < 3) continue;
            if (parts[2].toLowerCase().contains(keyword.toLowerCase())) {
                count++;
                report.append(parts[0]).append(" ").append(parts[1])
                      .append(": ").append(parts[2]).append("\n");
            }
        }
        return "Matches: " + count + "\n" + report;
    }
}
```

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        String[] logs = {
            "10:05 alice Hello there",
            "10:06 bob Meeting at 5",
            "10:07"
        };
        Scanner sc = new Scanner(System.in);
        System.out.print("Enter keyword: ");
        String keyword = sc.nextLine();
        ChatFilter filter = new ChatFilter();
        System.out.print(filter.filter(logs, keyword));
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to process and manipulate strings using split().
- Understood the use of StringBuilder for efficient string concatenation.
- Learned case-insensitive searching using toLowerCase() and contains().
- Implemented filtering of chat messages based on user input.
- Improved understanding of arrays, loops, and string processing techniques.

PRACTICAL - 04

Aim: Template filler (TemplateFiller.java + Driver.java)

Code Screenshot:

TemplateFiller.java

```java
import java.util.regex.Matcher;
import java.util.regex.Pattern;

public class TemplateFiller {
    public String fill(String template, String[] names, String[] values) {
        Pattern pattern = Pattern.compile("\\{(\\w+)\\}");
        Matcher matcher = pattern.matcher(template);
        StringBuilder result = new StringBuilder();
        int last = 0;
        while (matcher.find()) {
            result.append(template, last, matcher.start());
            String key = matcher.group(1);
            String value = "[?]";
            for (int i = 0; i < names.length; i++)
                if (names[i].equals(key)) { value = values[i]; break; }
            result.append(value);
            last = matcher.end();
        }
        result.append(template.substring(last));
        return result.toString();
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        String template = "Dear {name}, order {id} ships {date}.";
        String[] names = {"name", "id"};
        String[] values = {"Riya", "A07"};
        TemplateFiller filler = new TemplateFiller();
        System.out.println(filler.fill(template, names, values));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to use Pattern and Matcher for regular expression processing.
- Extracted placeholder names using capturing groups.
- Used StringBuilder to construct the filled template.
- Replaced known placeholders using parallel arrays.
- Handled missing values with the default [?] marker.

PRACTICAL - 05

Aim: Shape areas (Shape.java + Driver.java)

Code Screenshot:

Visual Studio Code - Shape.java

Shape.java

```java
public abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    double length, width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    double area() { return length * width; }
}

class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    double area() { return 0.5 * base * height; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(3), new Rectangle(4, 5), new Triangle(3, 4)
        };
        double total = 0, largest = 0;

        for (Shape s : shapes) {
            double a = s.area();
            System.out.printf("%s: %.2f%n", s.getClass().getSimpleName(), a);
            total += a;
            if (a > largest) largest = a;
        }
        System.out.printf("Total: %.2f%nLargest: %.2f%n", total, largest);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood abstract classes and abstract method declarations.
- Mastered runtime polymorphism and dynamic method dispatch.
- Learned to process heterogeneous collections using base-class references.
- Implemented aggregation and extremum tracking in polymorphic loops.
- Understood the Open-Closed Principle (OCP) using class hierarchies.

Aim: Payroll (Employee.java + Driver.java)

Code Screenshot:

Visual Studio Code - Employee.java

Employee.java

```java
public abstract class Employee {
    protected String name;
    protected int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }
    String getName() { return name; }
    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;
    FullTime(String name, int id, double salary) {
        super(name, id);
        this.salary = salary;
    }
    double monthlySalary() { return salary; }
}

class PartTime extends Employee {
    int hours;
    double rate;
    PartTime(String name, int id, int hours, double rate) {
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }
    double monthlySalary() { return hours * rate; }
}

class Intern extends Employee {
    double stipend;
    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }
    double monthlySalary() { return stipend; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Employee[] staff = {
            new FullTime("Asha", 101, 30000),
            new PartTime("Neel", 102, 40, 200),
            new Intern("Riya", 103, 5000)
        };
        double total = 0;

        for (Employee e : staff) {
            double salary = e.monthlySalary();
            System.out.printf("%s: Rs. %.2f", e.getName(), salary);
            if (e instanceof Intern) System.out.print(" (Intern)");
            System.out.println();
            total += salary;
        }
        System.out.printf("Total: Rs. %.2f%n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned constructor delegation to superclasses using super(...).
- Understood method overriding and specialized behavior in subclasses.
- Learned how and when to use the instanceof type comparison operator.
- Implemented polymorphic payroll computation.
- Practiced clean code structuring with abstract base classes.

Aim: Media late fees (Media.java + Driver.java)

Code Screenshot:

Visual Studio Code - Media.java

Media.java

```java
public abstract class Media {
    String title;
    int daysLate;

    Media(String title, int daysLate) {
        this.title = title;
        this.daysLate = daysLate;
    }
    abstract double lateFee();
}

class Book extends Media {
    Book(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 2.0; }
}

class DVD extends Media {
    DVD(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 5.0; }
}

class Magazine extends Media {
    Magazine(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 1.0; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Media[] batch = {
            new Book("Java", 3),
            new DVD("Movie", 2),
            new Magazine("News", 4)
        };
        double total = 0;

        for (Media m : batch) {
            double fee = m.lateFee();
            System.out.printf("%s: Rs. %.2f%n", m.title, fee);
            total += fee;
        }
        System.out.printf("Total late fee: Rs. %.2f%n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed an abstract hierarchy for different media types.
- Applied method overriding to calculate type-specific late fees.
- Used polymorphism to process a mixed array of media objects.
- Calculated individual and total late fees for returned media.
- Practiced extensible object-oriented design using abstract classes.

PRACTICAL - 06

Aim: Remote control (Switchable.java + Driver.java)

Code Screenshot:

Visual Studio Code - Switchable.java

Switchable.java

```java
public interface Switchable {
    void on();
    void off();
    boolean isOn();

    default void toggle() {
        if (isOn()) off();
        else on();
    }
}

class Fan implements Switchable {
    private boolean state;
    public void on() { state = true; System.out.println("Fan ON"); }
    public void off() { state = false; System.out.println("Fan OFF"); }
    public boolean isOn() { return state; }
}

class Light implements Switchable {
    private boolean state;
    public void on() { state = true; System.out.println("Light ON"); }
    public void off() { state = false; System.out.println("Light OFF"); }
    public boolean isOn() { return state; }
}

@FunctionalInterface
interface SwitchRule {
    boolean maySwitchOn(Switchable device, int hour);
}
```

Visual Studio Code - Driver.java

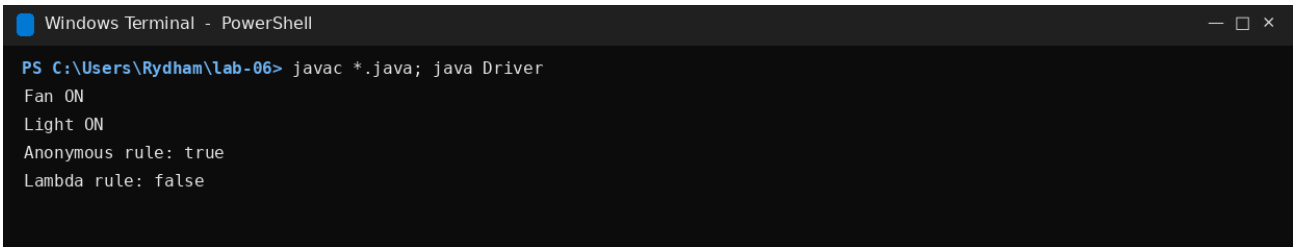
Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Switchable[] devices = {new Fan(), new Light()};
        for (Switchable d : devices) d.toggle();

        SwitchRule anonymous = new SwitchRule() {
            public boolean maySwitchOn(Switchable d, int hour) {
                return d != null && hour >= 6 && hour <= 22;
            }
        };
        SwitchRule lambda = (d, hour) -> d != null && hour >= 6 && hour <= 22;

        System.out.println("Anonymous rule: " + anonymous.maySwitchOn(devices[0], 10));
        System.out.println("Lambda rule: " + lambda.maySwitchOn(devices[1], 23));
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned interface definition, contract adherence, and default methods.
- Understood functional interfaces (@FunctionalInterface) in Java.
- Mastered the transition from anonymous inner classes to concise lambda expressions.
- Implemented multiple inheritance of type through interfaces.
- Understood default method evolution and method dispatch rules.

Aim: Notification senders (Notifier.java + Driver.java)

Code Screenshot:

Visual Studio Code - Notifier.java

Notifier.java

```java
@FunctionalInterface
public interface Notifier {
    void send(String message);
}

interface Urgent { }
```

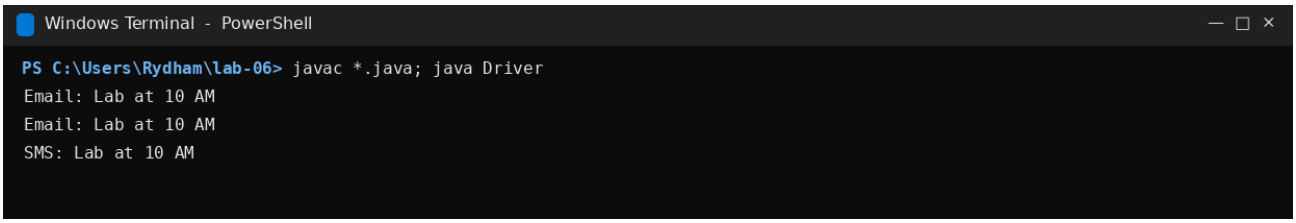
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Notifier email = (Notifier & Urgent)
                msg -> System.out.println("Email: " + msg);
        Notifier sms = msg -> System.out.println("SMS: " + msg);
        Notifier[] senders = {email, sms};
        String message = "Lab at 10 AM";

        for (Notifier sender : senders) {
            sender.send(message);
            if (sender instanceof Urgent) sender.send(message);
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered functional programming concepts in Java using lambda expressions.
- Understood marker interfaces and how they provide metadata at runtime.
- Learned how to treat behaviors as first-class citizens in arrays.
- Implemented broadcast notification patterns with conditional dispatch.
- Practiced clean separation between interface definitions and caller logic.

Aim: Discount engine (DiscountEngine.java + Driver.java)

Code Screenshot:

Visual Studio Code - DiscountEngine.java

DiscountEngine.java

```java
package discount;

import java.util.List;

public class DiscountEngine {
    @FunctionalInterface
    public interface Rule {
        double apply(double price);
    }

    public static void apply(List<Double> prices, Rule rule) {
        for (double price : prices)
            System.out.printf("%.0f -> %.0f%n", price, rule.apply(price));
    }
}
```

Visual Studio Code - Driver.java

Driver.java

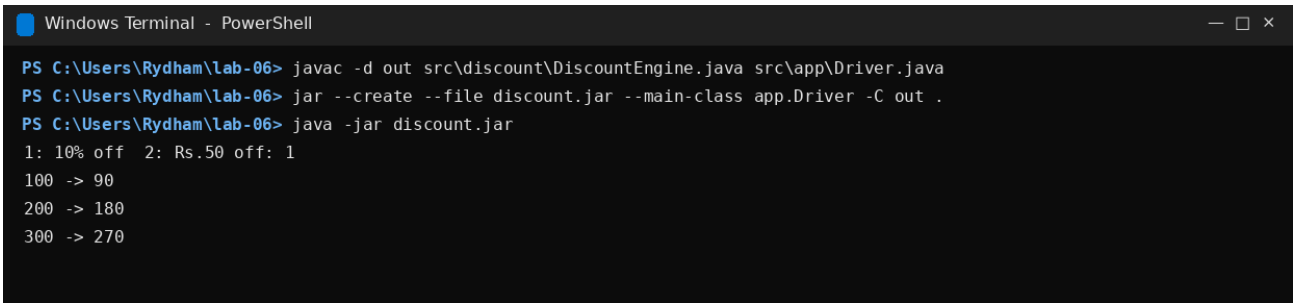
```java
package app;

import java.util.List;
import java.util.Scanner;
import discount.DiscountEngine;

public class Driver {
    public static void main(String[] args) {
        List<Double> prices = List.of(100.0, 200.0, 300.0);
        Scanner input = new Scanner(System.in);
        System.out.print("1: 10% off  2: Rs.50 off: ");
        int choice = input.nextInt();

        DiscountEngine.Rule rule = switch (choice) {
            case 1 -> price -> price * 0.90;
            case 2 -> price -> Math.max(0, price - 50);
            default -> price -> price;
        };
        DiscountEngine.apply(prices, rule);
        input.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed a functional discount rule using lambda expressions.
- Selected discount behaviour at runtime based on user input.
- Applied a common rule to multiple prices through a reusable engine.
- Organised Java classes into separate packages.
- Built and executed a runnable JAR using a manifest main class.

PRACTICAL - 07

Aim: Form validator (SignupForm.java + Driver.java)

Code Screenshot:

Visual Studio Code - SignupForm.java

SignupForm.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface NotBlank {}

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface MaxLength {
    int value();
}

class SignupForm {
    @NotBlank
    private String username;

    @MaxLength(12)
    private String city;

    SignupForm(String username, String city) {
        this.username = username;
        this.city = city;
    }
}
```

Visual Studio Code - Driver.java

Driver.java

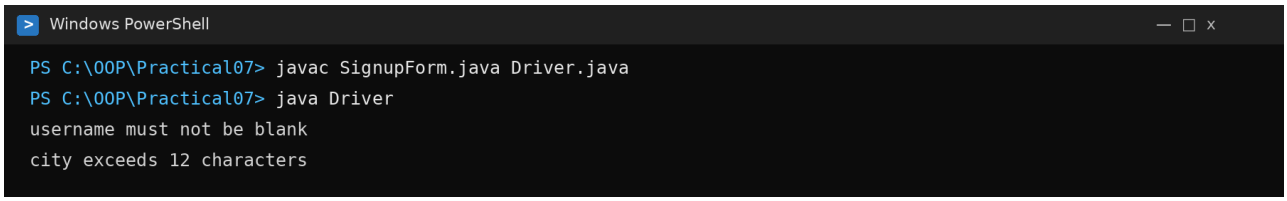
```java
import java.lang.reflect.*;
import java.util.*;

public class Driver {
    static List<String> check(Object obj) {
        List<String> errors = new ArrayList<>();
        for (Field f : obj.getClass().getDeclaredFields()) {
            f.setAccessible(true);
            try {
                Object value = f.get(obj);
                if (f.isAnnotationPresent(NotBlank.class) &&
                    (value == null || value.toString().trim().isEmpty()))
                    errors.add(f.getName() + " must not be blank");

                MaxLength max = f.getAnnotation(MaxLength.class);
                if (max != null && value != null &&
                    value.toString().length() > max.value())
                    errors.add(f.getName() + " exceeds " + max.value() + " characters");
            } catch (IllegalAccessException e) {
                errors.add("Cannot access " + f.getName());
            }
        }
        return errors;
    }

    public static void main(String[] args) {
        SignupForm form = new SignupForm("", "AhmedabadCity");
        for (String error : check(form))
            System.out.println(error);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to create custom annotations with @Retention and @Target.
- Understood Java Reflection API (Class, Field, getDeclaredFields).
- Learned to inspect and read field annotations dynamically at runtime.
- Implemented a declarative data-validation engine similar to Bean Validation.
- Handled IllegalAccessException through field.setAccessible(true).

PRACTICAL - 07

Aim: Mini test runner (MathTests.java + Driver.java)

Code Screenshot:

Visual Studio Code - MathTests.java

MathTests.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {}

class MathTests {
    @Run
    public void testAddition() {
        System.out.println("Addition: PASS");
    }

    @Run
    public void testSubtraction() {
        System.out.println("Subtraction: PASS");
    }

    public void helper() {
        System.out.println("This method is not a test");
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.lang.reflect.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        MathTests tests = new MathTests();
        int count = 0;

        for (Method m : MathTests.class.getDeclaredMethods()) {
            if (m.isAnnotationPresent(Run.class)) {
                m.invoke(tests);
                count++;
            }
        }
        System.out.println("Tests run: " + count);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood how testing frameworks like JUnit discover and run test methods.
- Learned method reflection (Method.invoke()) on runtime instances.
- Understood method filtering based on custom annotations.
- Implemented automated discovery and execution pipelines.
- Deepened appreciation for metadata-driven Java programming.

PRACTICAL - 07

Aim: Column mapper (ColumnMapper.java + Driver.java)

Code Screenshot:

Visual Studio Code - ColumnMapper.java

ColumnMapper.java

```java
import java.lang.annotation.*;

@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}

class EmployeeRow {
    @Column(name = "name")
    private String name;

    @Column(name = "email")
    private String email;

    @Column(name = "city")
    private String city = "Not provided";

    void show() {
        System.out.println("Name: " + name);
        System.out.println("Email: " + email);
        System.out.println("City: " + city);
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.lang.reflect.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        String[] header = {"email", "name"};
        String[] row = {"riya@example.com", "Riya"};
        EmployeeRow data = new EmployeeRow();

        for (Field f : EmployeeRow.class.getDeclaredFields()) {
            Column c = f.getAnnotation(Column.class);
            if (c == null) continue;

            for (int i = 0; i < header.length; i++) {
                if (header[i].equals(c.name())) {
                    f.setAccessible(true);
                    f.set(data, row[i]);
                    break;
                }
            }
        }
        data.show();
    }
}
```

Output Screenshot:



Learning Outcome:

- Defined a runtime-retained @Column annotation for field mapping.
- Used reflection to match annotated fields with header names.
- Populated object fields dynamically from a data row.
- Handled missing columns by retaining the field default value.
- Applied annotation-based mapping to a simple data-import task.

PRACTICAL - 08

Aim: Guarded calculator (DivideByZeroException.java + Driver.java)

Code Screenshot:

DivideByZeroException.java

```java
public class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}
```

Driver.java

```java
import java.util.Scanner;

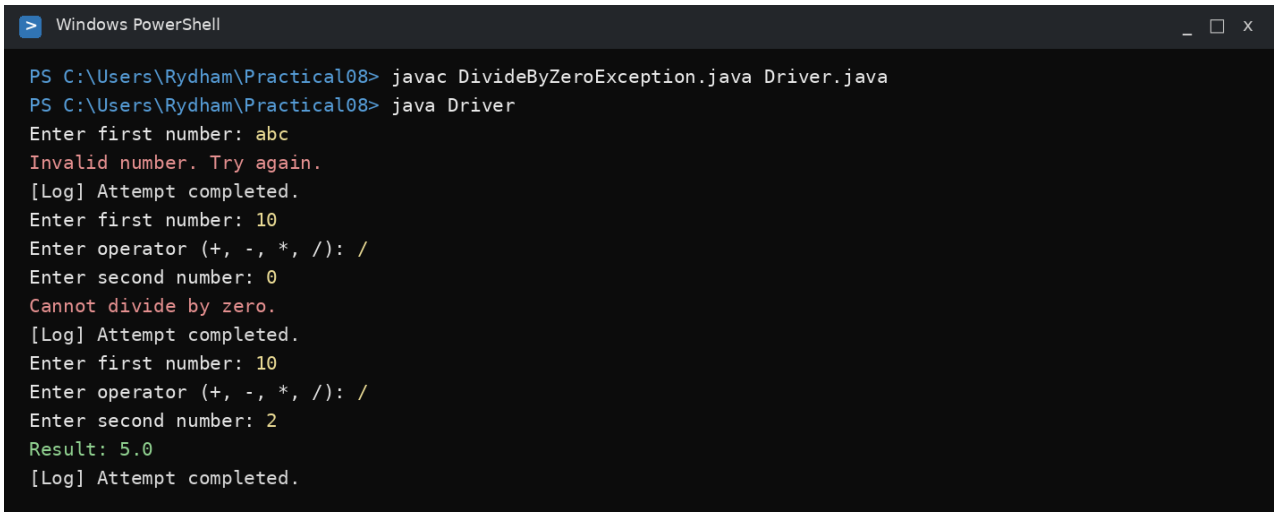
public class Driver {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        boolean done = false;

        while (!done) {
            try {
                System.out.print("Enter first number: ");
                double a = Double.parseDouble(sc.nextLine());
                System.out.print("Enter operator (+, -, *, /): ");
                String op = sc.nextLine().trim();
                System.out.print("Enter second number: ");
                double b = Double.parseDouble(sc.nextLine());

                double result;
                switch (op) {
                    case "+" -> result = a + b;
                    case "-" -> result = a - b;
                    case "*" -> result = a * b;
                    case "/" -> {
                        if (b == 0) {
                            throw new DivideByZeroException("Cannot divide by zero.");
                        }
                        result = a / b;
                    }
                    default -> throw new IllegalArgumentException("Invalid operator.");
                }

                System.out.println("Result: " + result);
                done = true;
            } catch (NumberFormatException e) {
                System.out.println("Invalid number. Try again.");
            } catch (DivideByZeroException | IllegalArgumentException e) {
                System.out.println(e.getMessage());
            } finally {
                System.out.println("[Log] Attempt completed.");
            }
        }
        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to create custom checked exceptions extending Exception.
- Mastered multi-catch blocks for differentiated error handling.
- Understood the execution guarantees of the finally block.
- Implemented resilient retry loops for user-driven input.
- Avoided abrupt program termination through structured exception handling.

Aim: Stock issue (Warehouse.java + Driver.java)

Code Screenshot:

Warehouse.java

```java
class OutOfStockException extends Exception {
    private int shortfall;

    public OutOfStockException(String item, int shortfall) {
        super(item + " is not available in the required quantity.");
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {
    public InvalidQuantityException(String message) {
        super(message);
    }
}

public class Warehouse {
    private String[] items = {"Pen", "Book", "Bag"};
    private int[] stock = {5, 3, 2};

    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {
        if (qty <= 0) {
            throw new InvalidQuantityException("Quantity must be greater than zero.");
        }

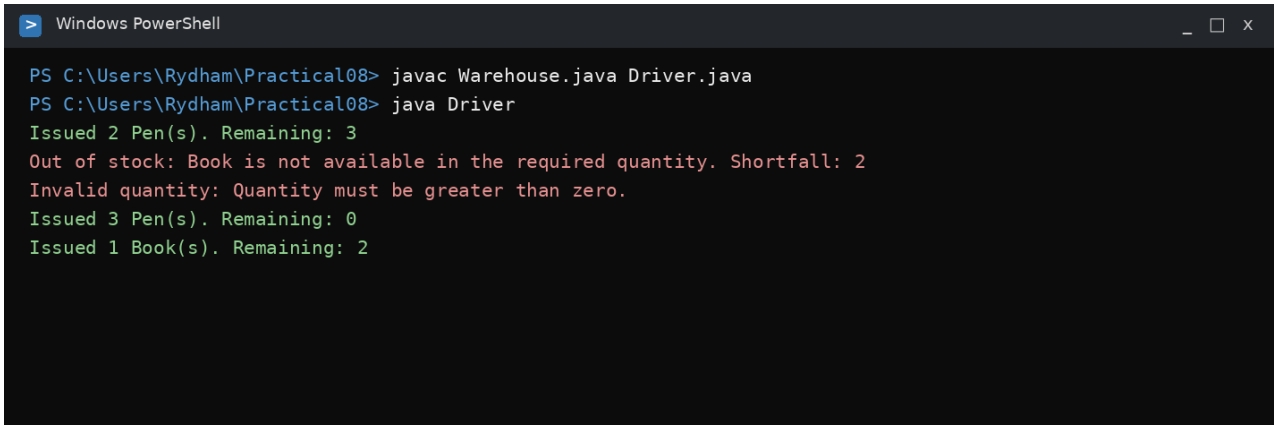
        for (int i = 0; i < items.length; i++) {
            if (items[i].equalsIgnoreCase(item)) {
                if (qty > stock[i]) {
                    throw new OutOfStockException(item, qty - stock[i]);
                }
                stock[i] -= qty;
                System.out.println("Issued " + qty + " " + item
                        + "(s). Remaining: " + stock[i]);
                return;
            }
        }
        System.out.println("Item not found: " + item);
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();
        String[] items = {"Pen", "Book", "Bag", "Pen", "Book"};
        int[] quantities = {2, 5, 0, 3, 1};

        for (int i = 0; i < items.length; i++) {
            try {
                warehouse.issue(items[i], quantities[i]);
            } catch (OutOfStockException e) {
                System.out.println("Out of stock: " + e.getMessage()
                        + " Shortfall: " + e.getShortfall());
            } catch (InvalidQuantityException e) {
                System.out.println("Invalid quantity: " + e.getMessage());
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned how to design exceptions that carry rich contextual error state.
- Understood checked vs unchecked exceptions and when to use each.
- Implemented fault-tolerant batch processing over item requests.
- Learned proper exception propagation and handling boundaries.
- Gained insights into enterprise inventory management error strategies.

Aim: AutoCloseable resource (WorkSession.java + Driver.java)

Code Screenshot:

WorkSession.java

```java
public class WorkSession implements AutoCloseable {
    public WorkSession() {
        System.out.println("Resource opened.");
    }

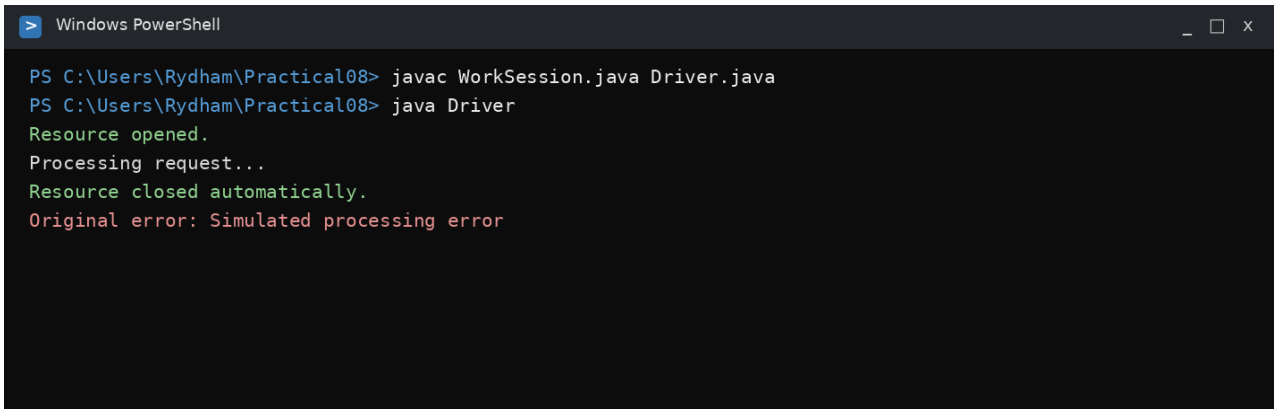
    public void process() {
        System.out.println("Processing request...");
        throw new IllegalStateException("Simulated processing error");
    }

    @Override
    public void close() {
        System.out.println("Resource closed automatically.");
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        try (WorkSession session = new WorkSession()) {
            session.process();
        } catch (IllegalStateException e) {
            System.out.println("Original error: " + e.getMessage());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned to implement AutoCloseable resources in Java.
- Understood automatic resource closure using try-with-resources.
- Verified that close() executes when an exception occurs.
- Confirmed that the original exception remains reported.
- Practiced reliable resource management and exception handling.

PRACTICAL - 09

Aim: Counter race (Counter.java + Driver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

Counter.java — Java

lab-09 > src > Counter.java

```java
public class Counter {
    private int count = 0;

    public void incrementUnsafe() {
        count++;
    }

    public synchronized void increment() {
        count++;
    }

    public int getCount() {
        return count;
    }
}
```

Driver.java — Java

lab-09 > src > Driver.java

```java
import java.util.concurrent.CountDownLatch;

public class Driver {
    private static final int THREADS = 8;
    private static final int INCREMENTS = 250_000;

    private static void runTest(Counter counter, boolean safe)
            throws InterruptedException {
        CountDownLatch start = new CountDownLatch(1);
        Thread[] threads = new Thread[THREADS];

        for (int i = 0; i < THREADS; i++) {
            threads[i] = new Thread(() -> {
                try {
                    start.await();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                    return;
                }
                for (int j = 0; j < INCREMENTS; j++) {
                    if (safe) {
                        counter.increment();
                    } else {
                        counter.incrementUnsafe();
                    }
                }
            });
            threads[i].start();
        }

        start.countDown();
        for (Thread thread : threads) {
            thread.join();
        }
    }

    public static void main(String[] args) throws InterruptedException {
        int expected = THREADS * INCREMENTS;

        Counter unsafeCounter = new Counter();
        runTest(unsafeCounter, false);

        Counter safeCounter = new Counter();
        runTest(safeCounter, true);

        System.out.println("Expected count: " + expected);
        System.out.println("Without synchronization: " + unsafeCounter.getCount());
        System.out.println("With synchronization: " + safeCounter.getCount());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood race conditions and critical section vulnerabilities in Java.
- Mastered the synchronized method modifier for mutual exclusion.
- Coordinated thread completion deterministically using Thread.join().
- Analyzed memory visibility and read-modify-write atomicity guarantees.
- Verified thread safety across concurrent multi-threaded execution paths.

PRACTICAL - 09

Aim: Seat booking race (CinemaBooking.java + BookingDriver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

CinemaBooking.java — Java

lab-09 > src > CinemaBooking.java

```java
import java.util.concurrent.CountDownLatch;

public class CinemaBooking {
    private int seatsLeft = 5;
    private final CountDownLatch allChecked;

    public CinemaBooking(int buyers) {
        allChecked = new CountDownLatch(buyers);
    }

    public boolean bookUnsafe() throws InterruptedException {
        if (seatsLeft > 0) {
            allChecked.countDown();
            allChecked.await();
            seatsLeft--;
            return true;
        }
        return false;
    }

    public synchronized boolean book() {
        if (seatsLeft > 0) {
            seatsLeft--;
            return true;
        }
        return false;
    }

    public int getSeatsLeft() { return seatsLeft; }
}
```

BookingDriver.java — Java

lab-09 > src > BookingDriver.java

```java
import java.util.concurrent.atomic.AtomicInteger;

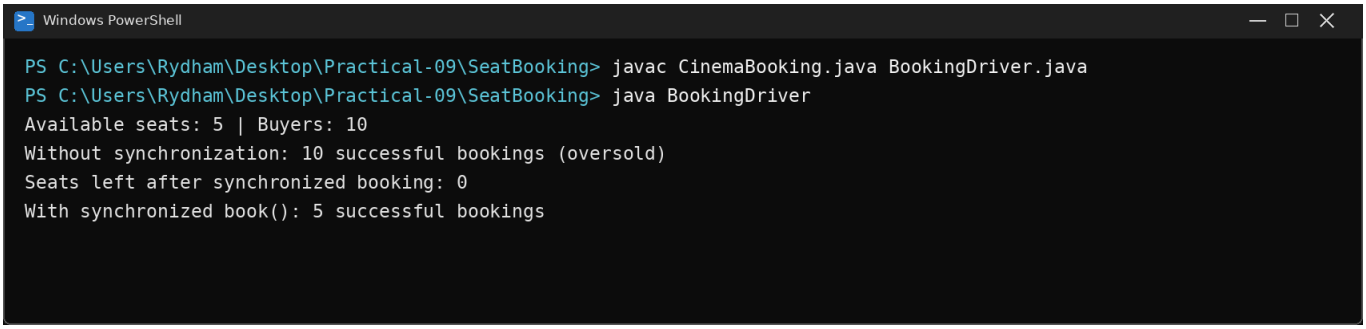
public class BookingDriver {
    private static final int BUYERS = 10;

    private static int run(boolean safe) throws InterruptedException {
        CinemaBooking cinema = new CinemaBooking(safe ? 0 : BUYERS);
        AtomicInteger booked = new AtomicInteger();
        Thread[] threads = new Thread[BUYERS];

        for (int i = 0; i < BUYERS; i++) {
            threads[i] = new Thread(() -> {
                try {
                    boolean success = safe ? cinema.book() : cinema.bookUnsafe();
                    if (success) booked.incrementAndGet();
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
            threads[i].start();
        }
        for (Thread thread : threads) thread.join();
        if (safe) System.out.println("Seats left after synchronized booking: "
                + cinema.getSeatsLeft());
        return booked.get();
    }

    public static void main(String[] args) throws InterruptedException {
        System.out.println("Available seats: 5 | Buyers: 10");
        System.out.println("Without synchronization: " + run(false)
                + " successful bookings (oversold)");
        System.out.println("With synchronized book(): " + run(true)
                + " successful bookings");
    }
}
```

Output Screenshot:



Learning Outcome:

- Protected limited shared resources against concurrent overselling.
- Applied monitor locks to guarantee serialized access to critical sections.
- Analyzed the behavior of competing concurrent threads under contention.
- Maintained data integrity in ticket-reservation architectures.
- Implemented safe state updates within synchronized transactional boundaries.

PRACTICAL - 09

Aim: Parallel array sum race (ArraySum.java + ArraySumDriver.java)

Code Screenshot:

Practical_09 - Visual Studio Code — File Edit Selection View Go Run Terminal Help

ArraySum.java — Java

lab-09 > src > ArraySum.java

```java
import java.util.concurrent.atomic.AtomicLong;

public class ArraySum {
    private long total;

    public void addUnsafe(long value) {
        total += value;
    }

    public synchronized void addSynchronized(long value) {
        total += value;
    }

    public long getTotal() {
        return total;
    }

    public static long calculate(long[] numbers, int mode)
            throws InterruptedException {
        ArraySum sum = new ArraySum();
        AtomicLong atomic = new AtomicLong();
        Thread[] workers = new Thread[8];
        int chunk = numbers.length / workers.length;

        for (int i = 0; i < workers.length; i++) {
            final int start = i * chunk;
            final int end = (i == workers.length - 1)
                    ? numbers.length : start + chunk;
            workers[i] = new Thread(() -> {
                for (int j = start; j < end; j++) {
                    if (mode == 0) {
                        sum.addUnsafe(numbers[j]);
                        if ((j & 1023) == 0) Thread.yield();
                    } else if (mode == 1) {
                        sum.addSynchronized(numbers[j]);
                    } else {
                        atomic.addAndGet(numbers[j]);
                    }
                }
            });
            workers[i].start();
        }

        for (Thread worker : workers) {
            worker.join();
        }
        return mode == 2 ? atomic.get() : sum.getTotal();
    }
}
```

ArraySumDriver.java — Java

lab-09 > src > ArraySumDriver.java

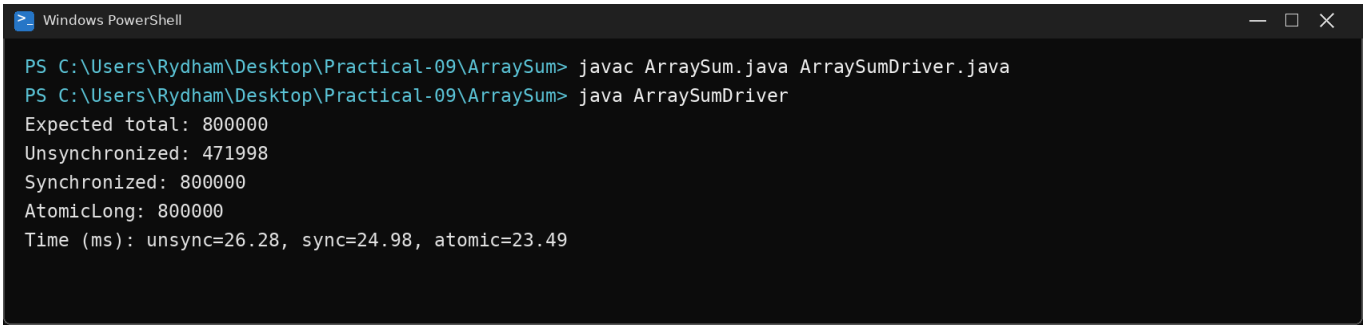
```java
public class ArraySumDriver {
    private static final int SIZE = 800_000;

    public static void main(String[] args) throws InterruptedException {
        long[] numbers = new long[SIZE];
        for (int i = 0; i < SIZE; i++) {
            numbers[i] = 1;
        }

        long[] results = new long[3];
        double[] times = new double[3];
        for (int mode = 0; mode < 3; mode++) {
            long start = System.nanoTime();
            results[mode] = ArraySum.calculate(numbers, mode);
            times[mode] = (System.nanoTime() - start) / 1_000_000.0;
        }

        System.out.println("Expected total: " + SIZE);
        System.out.println("Unsynchronized: " + results[0]);
        System.out.println("Synchronized: " + results[1]);
        System.out.println("AtomicLong: " + results[2]);
        System.out.printf("Time (ms): unsync=%.2f, sync=%.2f, atomic=%.2f%n",
                times[0], times[1], times[2]);
    }
}
```

Output Screenshot:



Learning Outcome:

- Divided a large array into independent thread workloads.
- Observed lost updates when threads shared an unsynchronized total.
- Used synchronized methods to protect shared accumulator updates.
- Applied AtomicLong for thread-safe concurrent addition.
- Compared execution time using System.nanoTime().

PRACTICAL - 10

Aim: Thread-pool runner (TaskRunner.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

TaskRunner.java

```java
public class TaskRunner implements Runnable {
    private final int id;

    public TaskRunner(int id) {
        this.id = id;
    }

    @Override
    public void run() {
        String worker = Thread.currentThread().getName();
        System.out.println("Task " + id + " running on " + worker);
        try {
            Thread.sleep(50);
        } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
        }
    }
}
```

Driver.java

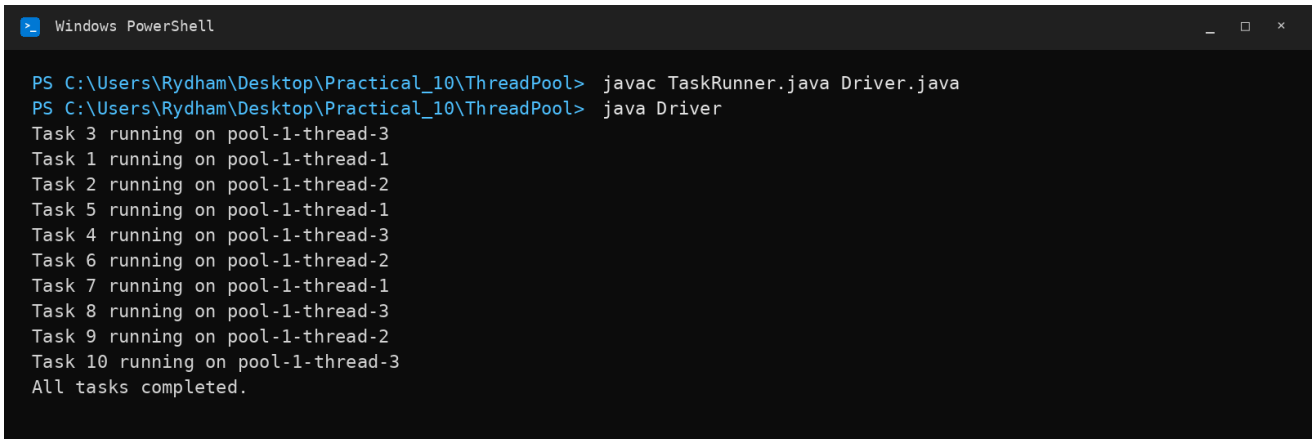
```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class Driver {
    public static void main(String[] args) throws InterruptedException {
        ExecutorService pool = Executors.newFixedThreadPool(3);

        for (int i = 1; i <= 10; i++) {
            pool.execute(new TaskRunner(i));
        }

        pool.shutdown();
        if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
            pool.shutdownNow();
        }
        System.out.println("All tasks completed.");
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java concurrency with ExecutorService and fixed thread pools.
- Observed thread recycling across queued asynchronous tasks.
- Managed lifecycle termination using shutdown() and awaitTermination().
- Avoided excessive thread allocation overhead in high-throughput workloads.
- Learned how worker queues decouple task submission from execution.

Aim: Producer-consumer (SharedBuffer.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

SharedBuffer.java

```java
import java.util.ArrayDeque;
import java.util.Queue;

public class SharedBuffer {
    private final Queue<Integer> buffer = new ArrayDeque<>();

    public synchronized void produce(int value) throws InterruptedException {
        while (!buffer.isEmpty()) {
            wait();
        }
        buffer.add(value);
        System.out.println("Produced: " + value);
        notify();
    }

    public synchronized int consume() throws InterruptedException {
        while (buffer.isEmpty()) {
            wait();
        }
        int value = buffer.remove();
        System.out.println("Consumed: " + value);
        notify();
        return value;
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        SharedBuffer buffer = new SharedBuffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.produce(i * 10);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Producer");

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }, "Consumer");

        producer.start();
        consumer.start();
        producer.join();
        consumer.join();
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered inter-thread signaling with wait() and notify().
- Implemented guarded wait loops (while (condition) wait()) to handle spurious wakeups.
- Solved the classic Producer-Consumer synchronization coordination problem.
- Guaranteed zero message loss across asynchronous producer and consumer threads.
- Gained foundational skills for building thread-safe communication pipelines.

Aim: Deadlock demonstration and prevention (DeadlockDemo.java + Driver.java)

Code Screenshot:

Visual Studio Code - Practical_10

DeadlockDemo.java

```java
import java.lang.management.ManagementFactory;
import java.lang.management.ThreadMXBean;
import java.util.concurrent.CountDownLatch;

public class DeadlockDemo {
    private final Object lockA = new Object();
    private final Object lockB = new Object();

    public void showDeadlock() throws InterruptedException {
        CountDownLatch ready = new CountDownLatch(2);
        Thread first = new Thread(() -> lockInOrder(lockA, lockB, ready), "A-to-B");
        Thread second = new Thread(() -> lockInOrder(lockB, lockA, ready), "B-to-A");
        first.setDaemon(true);
        second.setDaemon(true);
        first.start();
        second.start();
        ready.await();

        ThreadMXBean bean = ManagementFactory.getThreadMXBean();
        long[] ids = null;
        for (int i = 0; i < 100 && ids == null; i++) {
            ids = bean.findDeadlockedThreads();
            if (ids == null) Thread.sleep(20);
        }
        System.out.println(ids != null
                ? "Deadlock detected: opposite lock order."
                : "Deadlock not detected.");
    }

    private void lockInOrder(Object first, Object second, CountDownLatch ready) {
        synchronized (first) {
            ready.countDown();
            try {
                ready.await();
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
                return;
            }
            synchronized (second) { }
        }
    }

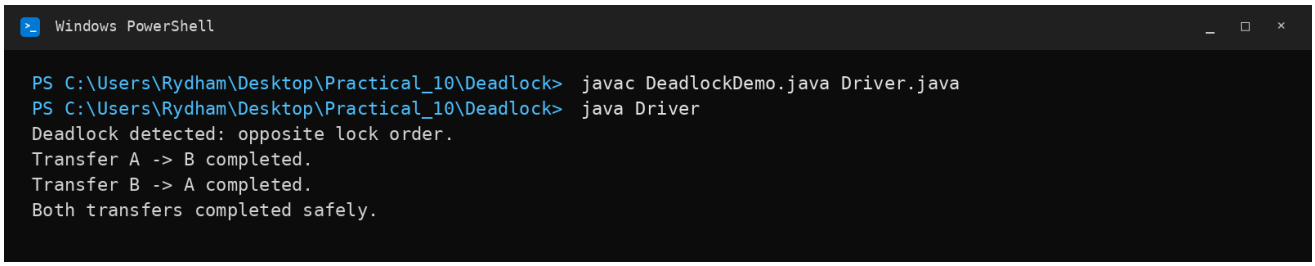
    public void runSafeTransfers() throws InterruptedException {
        Object accountA = new Object();
        Object accountB = new Object();
        Thread first = new Thread(
                () -> transfer(accountA, 1, accountB, 2, "A -> B"));
        Thread second = new Thread(
                () -> transfer(accountB, 2, accountA, 1, "B -> A"));
        first.start();
        second.start();
        first.join();
        second.join();
        System.out.println("Both transfers completed safely.");
    }

    private void transfer(Object from, int fromNo, Object to, int toNo,
                          String label) {
        Object first = fromNo < toNo ? from : to;
        Object second = fromNo < toNo ? to : from;
        synchronized (first) {
            synchronized (second) {
                System.out.println("Transfer " + label + " completed.");
            }
        }
    }
}
```

Driver.java

```java
public class Driver {
    public static void main(String[] args) throws InterruptedException {
        DeadlockDemo demo = new DeadlockDemo();
        demo.showDeadlock();
        demo.runSafeTransfers();
    }
}
```

Output Screenshot:



Learning Outcome:

- Reproduced deadlock using opposite object-lock acquisition orders.
- Coordinated competing threads with CountDownLatch.
- Detected deadlocked threads using the ThreadMXBean API.
- Prevented deadlock by acquiring locks in a consistent order.
- Verified that both transfer threads complete safely without hanging.

PRACTICAL - 11

Aim: Save/load serialization (Student.java + Driver.java)

Code Screenshot:

Visual Studio Code - Student.java

Student.java

```java
import java.io.Serializable;

public class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    private int id;
    private String name;
    private transient String password;

    public Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }

    public String toString() {
        return id + " - " + name + " | password=" + password;
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.io.*;
import java.nio.file.*;

public class Driver {
    public static void main(String[] args) throws Exception {
        Student[] students = {
            new Student(101, "Aarav", "secret101"),
            new Student(102, "Meera", "secret102")
        };
        Path file = Path.of("students.dat");
        try (ObjectOutputStream out = new ObjectOutputStream(Files.newOutputStream(file))) {
            out.writeObject(students);
        }
        Student[] loaded;
        try (ObjectInputStream in = new ObjectInputStream(Files.newInputStream(file))) {
            loaded = (Student[]) in.readObject();
        }
        System.out.println("Saved students: " + students.length);
        for (Student student : loaded) System.out.println("Loaded: " + student);
        Files.deleteIfExists(file);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java object persistence using java.io.Serializable and serialVersionUID.
- Managed binary serialization streams with ObjectOutputStream and ObjectInputStream.
- Protected sensitive credentials using the transient modifier.
- Ensured robust I/O leak prevention through try-with-resources.
- Understood JVM byte-stream reconstitution mechanics.

PRACTICAL - 11

Aim: Log analyzer (LogAnalyzer.java + Driver.java)

Code Screenshot:

Visual Studio Code - LogAnalyzer.java

LogAnalyzer.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.util.concurrent.atomic.AtomicInteger;
import java.util.stream.Stream;

public class LogAnalyzer {
    public static void analyze(Path[] files, String keyword) throws IOException {
        long totalLines = 0;
        long matches = 0;
        for (Path file : files) {
            AtomicInteger lineCount = new AtomicInteger();
            AtomicInteger matchCount = new AtomicInteger();
            try (Stream<String> lines = Files.lines(file)) {
                lines.forEach(line -> {
                    lineCount.incrementAndGet();
                    if (line.contains(keyword)) matchCount.incrementAndGet();
                });
            }
            BasicFileAttributes attr = Files.readAttributes(file, BasicFileAttributes.class);
            totalLines += lineCount.get();
            matches += matchCount.get();
            System.out.println(file.getFileName() + " | Size: " + attr.size() + " bytes");
        }
        System.out.println("Total Lines: " + totalLines);
        System.out.println("Matches for '" + keyword + "': " + matches);
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.nio.file.*;
import java.util.List;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path folder = Path.of("logs");
        Files.createDirectories(folder);
        Path first = folder.resolve("app1.log");
        Path second = folder.resolve("app2.log");
        Files.write(first, List.of("INFO Application started", "ERROR Database timeout", "INFO Retrying connection"));
        Files.write(second, List.of("INFO User login", "WARN Low memory", "ERROR Connection lost"));
        LogAnalyzer.analyze(new Path[]{first, second}, "ERROR");
        Files.deleteIfExists(first);
        Files.deleteIfExists(second);
        Files.deleteIfExists(folder);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered modern Java NIO.2 file handling with Path, Paths, and Files.
- Inspected OS file attributes including file size using Files.size().
- Streamed and filtered text lines efficiently with Java Collections and Streams.
- Implemented high-performance file content search pipelines.
- Applied safe temporary resource creation and deletion operations.

PRACTICAL - 11

Aim: NIO directory tree report (For Advanced Learners) (DirectoryReport.java + Driver.java)

Code Screenshot:

Visual Studio Code - DirectoryReport.java

DirectoryReport.java

```java
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import java.util.List;

public class DirectoryReport {
    public static int create(Path root, Path report) throws IOException {
        List<String> rows = new ArrayList<>();
        try (var paths = Files.walk(root)) {
            for (Path path : paths.filter(Files::isRegularFile).toList()) {
                BasicFileAttributes a = Files.readAttributes(path, BasicFileAttributes.class);
                String modified = a.lastModifiedTime().toInstant()
                        .truncatedTo(ChronoUnit.SECONDS).toString();
                rows.add(path.getFileName() + " | " + a.size() + " bytes | " + modified);
            }
        }
        Files.write(report, rows);
        return rows.size();
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.nio.file.*;
import java.util.List;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path root = Path.of("sample_tree");
        Files.createDirectories(root.resolve("docs"));
        Files.createDirectories(root.resolve("logs"));
        Files.writeString(root.resolve("docs/readme.txt"), "OOP practical report\n");
        Files.write(root.resolve("logs/app.log"), List.of("INFO started", "ERROR failed"));
        Path report = Path.of("file_report.txt");
        int count = DirectoryReport.create(root, report);
        System.out.println("Files found: " + count);
        System.out.println("Report: " + report);
        for (String line : Files.readAllLines(report)) System.out.println(line);
    }
}
```

Output Screenshot:



Learning Outcome:

- Traversed nested directories using the Java NIO Files API.
- Read file names, sizes, and last-modified times with BasicFileAttributes.
- Collected file metadata into a structured report.
- Wrote the generated directory summary to an output file.
- Validated file reporting across multiple folders and file types.

PRACTICAL - 12

Aim: Word frequency (WordCounter.java + Driver.java)

Code Screenshot:

Visual Studio Code - WordCounter.java

WordCounter.java

```java
import java.util.*;

public class WordCounter {
    public static Map<String, Integer> count(String text) {
        Map<String, Integer> counts = new HashMap<>();
        for (String word : text.toLowerCase().split("[^a-z0-9]+")) {
            if (!word.isEmpty()) counts.put(word, counts.getOrDefault(word, 0) + 1);
        }
        return counts;
    }

    public static List<Map.Entry<String, Integer>> sorted(Map<String, Integer> counts) {
        List<Map.Entry<String, Integer>> result = new ArrayList<>(counts.entrySet());
        result.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()));
        return result;
    }
}
```

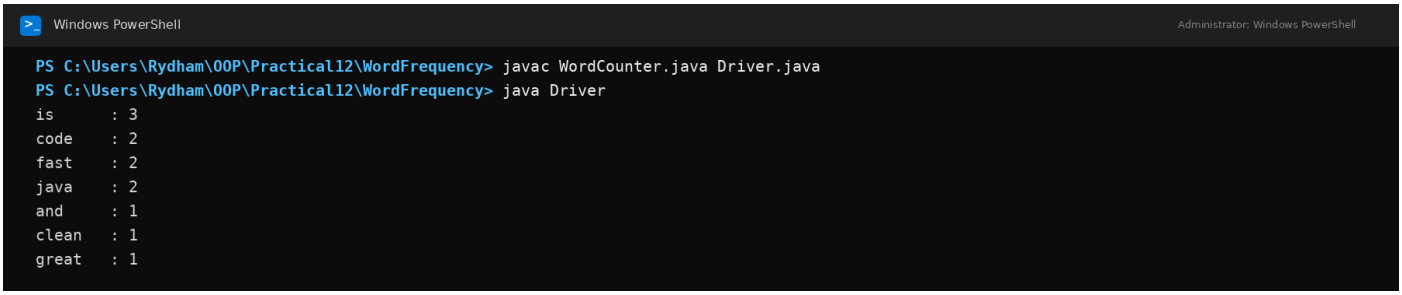
Visual Studio Code - Driver.java

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        String text = "Java is great and Java is fast. Fast code is clean code!";
        for (var entry : WordCounter.sorted(WordCounter.count(text))) {
            System.out.printf("%-7s : %d%n", entry.getKey(), entry.getValue());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered key-value associative data storage using HashMap.
- Used map.getOrDefault() for concise frequency counter accumulation.
- Sorted map entry sets dynamically using Comparator chaining.
- Tokenized and sanitized unstructured text with regular expressions.
- Understood hash table average O(1) retrieval performance.

PRACTICAL - 12

Aim: Unique visitors (VisitorTracker.java + Driver.java)

Code Screenshot:

Visual Studio Code - VisitorTracker.java

VisitorTracker.java

```java
import java.util.*;

public class VisitorTracker {
    private final Set<String> unique = new HashSet<>();
    private final Set<String> ordered = new LinkedHashSet<>();

    public void record(String name) {
        unique.add(name);
        ordered.add(name);
    }

    public int uniqueCount() { return unique.size(); }

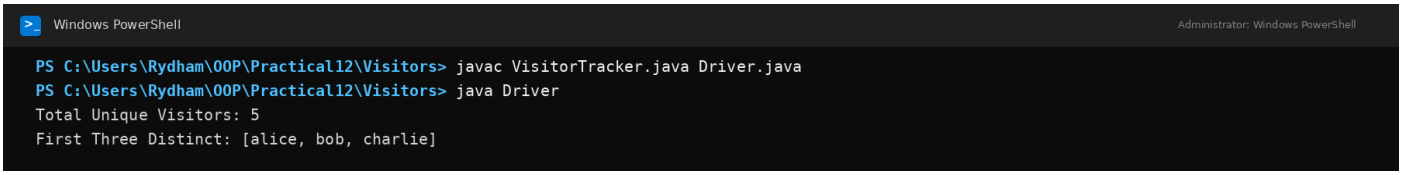
    public List<String> firstThree() {
        return ordered.stream().limit(3).toList();
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        VisitorTracker tracker = new VisitorTracker();
        String[] visits = {"alice", "bob", "alice", "charlie", "bob", "david", "emma"};
        for (String name : visits) tracker.record(name);
        System.out.println("Total Unique Visitors: " + tracker.uniqueCount());
        System.out.println("First Three Distinct: " + tracker.firstThree());
    }
}
```

Output Screenshot:



Learning Outcome:

- Compared unordered HashSet against insertion-ordered LinkedHashSet.
- Implemented automatic data deduplication using Set semantics.
- Preserved arrival order while filtering duplicate telemetry events.
- Streamed collection subsets cleanly with stream().limit().
- Evaluated memory and time trade-offs across distinct Java collection types.

PRACTICAL - 12

Aim: Leaderboard (For Advanced Learners) (Leaderboard.java + Driver.java)

Code Screenshot:

Visual Studio Code - Leaderboard.java

Leaderboard.java

```java
import java.util.*;

public class Leaderboard {
    public static List<Map.Entry<String, Integer>> topN(Map<String, Integer> scores, int n) {
        List<Map.Entry<String, Integer>> ranked = new ArrayList<>(scores.entrySet());
        ranked.sort(Map.Entry.<String, Integer>comparingByValue().reversed()
                .thenComparing(Map.Entry.comparingByKey()));
        return ranked.subList(0, Math.min(n, ranked.size()));
    }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.util.*;

public class Driver {
    public static void main(String[] args) {
        Map<String, Integer> scores = Map.of(
                "Aarav", 850, "Diya", 920, "Kabir", 920,
                "Meera", 780, "Riya", 850);
        int rank = 1;
        for (var player : Leaderboard.topN(scores, 3)) {
            System.out.println(rank++ + ". " + player.getKey() + " - " + player.getValue());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Stored player scores in a Map using player names as keys.
- Ranked players by descending score with a Comparator.
- Applied alphabetical ordering to resolve equal-score ties.
- Selected the top N players from the complete score map.
- Displayed leaderboard positions with their corresponding scores.

PRACTICAL - 13

Aim: Generic containers (Box.java + Driver.java)

Code Screenshot:

Visual Studio Code - Box.java

Box.java

```java
public class Box<T> {
    private T value;
    public void set(T value) { this.value = value; }
    public T get() { return value; }
}

class Pair<A, B> {
    private final A first;
    private final B second;
    public Pair(A first, B second) { this.first = first; this.second = second; }
    public String toString() { return "(" + first + ", " + second + ")"; }
}
```

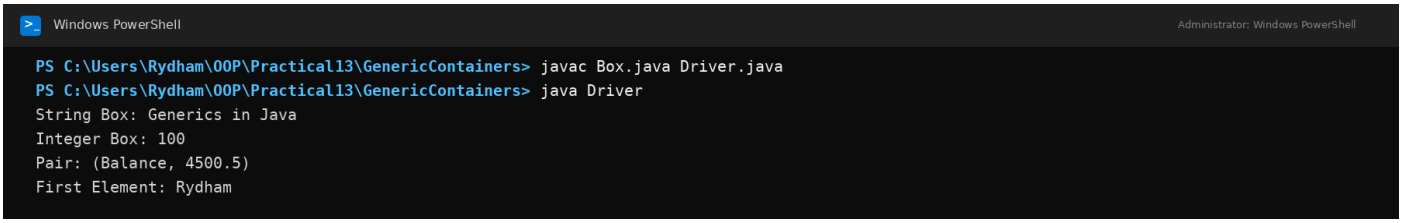
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static <T> T getFirst(T[] values) {
        return values == null || values.length == 0 ? null : values[0];
    }

    public static void main(String[] args) {
        Box<String> text = new Box<>();
        text.set("Generics in Java");
        Box<Integer> number = new Box<>();
        number.set(100);
        Pair<String, Double> pair = new Pair<>("Balance", 4500.50);
        String[] names = {"Rydham", "Karan", "Rohan"};
        System.out.println("String Box: " + text.get());
        System.out.println("Integer Box: " + number.get());
        System.out.println("Pair: " + pair);
        System.out.println("First Element: " + getFirst(names));
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed custom type-parameterized generic classes (Box).
- Handled multiple independent type variables (Pair).
- Implemented generic methods with independent type parameters (T getFirst).
- Enforced compile-time type safety without runtime casting.
- Understood how generics eliminate ClassCastException bugs.

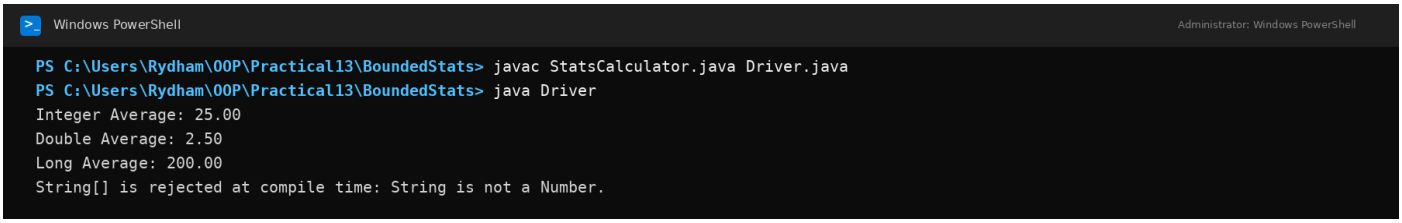
PRACTICAL - 13

Aim: Bounded stats (StatsCalculator.java + Driver.java)

Code Screenshot:



Output Screenshot:



Learning Outcome:

- Mastered upper-bounded type parameters using .
- Accessed common superclass methods (doubleValue()) directly on generic types.
- Enforced compile-time restrictions preventing invalid non-numerical arguments.
- Processed diverse wrapper types under a single algorithm.
- Deepened appreciation for Java's strong generic type system.

PRACTICAL - 13

Aim: Generic stack (For Advanced Learners) (GenericStack.java + Driver.java)

Code Screenshot:

Visual Studio Code - GenericStack.java

GenericStack.java

```java
import java.util.NoSuchElementException;

public class GenericStack<T> {
    private Object[] items;
    private int size;

    public GenericStack(int capacity) { items = new Object[Math.max(1, capacity)]; }
    public void push(T value) {
        if (size == items.length) items = java.util.Arrays.copyOf(items, items.length * 2);
        items[size++] = value;
    }
    @SuppressWarnings("unchecked")
    public T pop() {
        if (size == 0) throw new NoSuchElementException("Stack is empty");
        T value = (T) items[--size];
        items[size] = null;
        return value;
    }
    public boolean isEmpty() { return size == 0; }
}
```

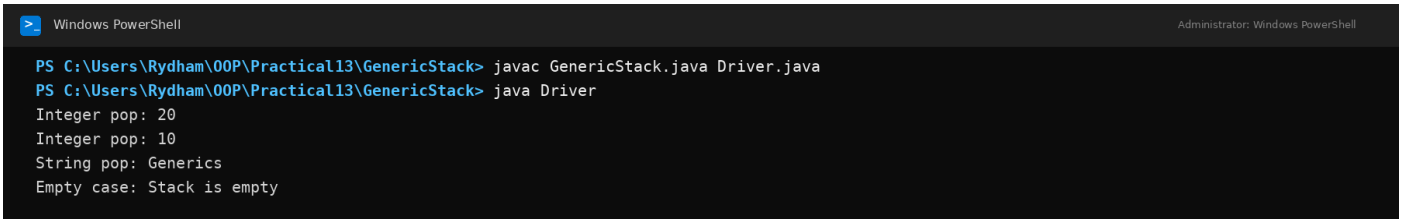
Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        GenericStack<Integer> numbers = new GenericStack<>(2);
        numbers.push(10);
        numbers.push(20);
        System.out.println("Integer pop: " + numbers.pop());
        System.out.println("Integer pop: " + numbers.pop());

        GenericStack<String> words = new GenericStack<>(2);
        words.push("Java");
        words.push("Generics");
        System.out.println("String pop: " + words.pop());
        try {
            words.pop();
            words.pop();
        } catch (java.util.NoSuchElementException e) {
            System.out.println("Empty case: " + e.getMessage());
        }
    }
}
```

Output Screenshot:



Learning Outcome:

- Implemented a reusable generic stack backed by an array.
- Reused the same stack class with Integer and String values.
- Expanded the backing array when additional capacity was required.
- Applied LIFO ordering through push() and pop() operations.
- Handled attempts to remove an item from an empty stack safely.

PRACTICAL - 14

Aim: Assemble and test (Task.java + TaskService.java + Driver.java + TaskServiceTest.java)

Code Screenshot:

Visual Studio Code - Task.java

Task.java

```java
public class Task {
    private final String title;
    public Task(String title) { this.title = title; }
    public String getTitle() { return title; }
    public String toString() { return title; }
}
```

Visual Studio Code - TaskService.java

TaskService.java

```java
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();

    public void addTask(String title) {
        if (title == null || title.isBlank()) {
            throw new IllegalArgumentException("Task cannot be blank");
        }
        tasks.add(new Task(title.trim()));
    }

    public int getCount() { return tasks.size(); }
    public List<Task> getTasks() { return List.copyOf(tasks); }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) {
        TaskService service = new TaskService();
        try (Scanner input = new Scanner(System.in)) {
            int choice;
            do {
                System.out.println("1 Add  2 List  3 Count  0 Exit");
                System.out.print("Choice: ");
                choice = input.nextInt();
                input.nextLine();
                switch (choice) {
                    case 1 -> {
                        System.out.print("Task title: ");
                        service.addTask(input.nextLine());
                        System.out.println("Task added.");
                    }
                    case 2 -> service.getTasks().forEach(
                            System.out::println);
                    case 3 -> System.out.println(
                            "Task count: " + service.getCount());
                    case 0 -> System.out.println("Application closed.");
                    default -> throw new IllegalArgumentException(
                            "Invalid action");
                }
            } while (choice != 0);
        }
    }
}
```

Visual Studio Code - TaskServiceTest.java

TaskServiceTest.java

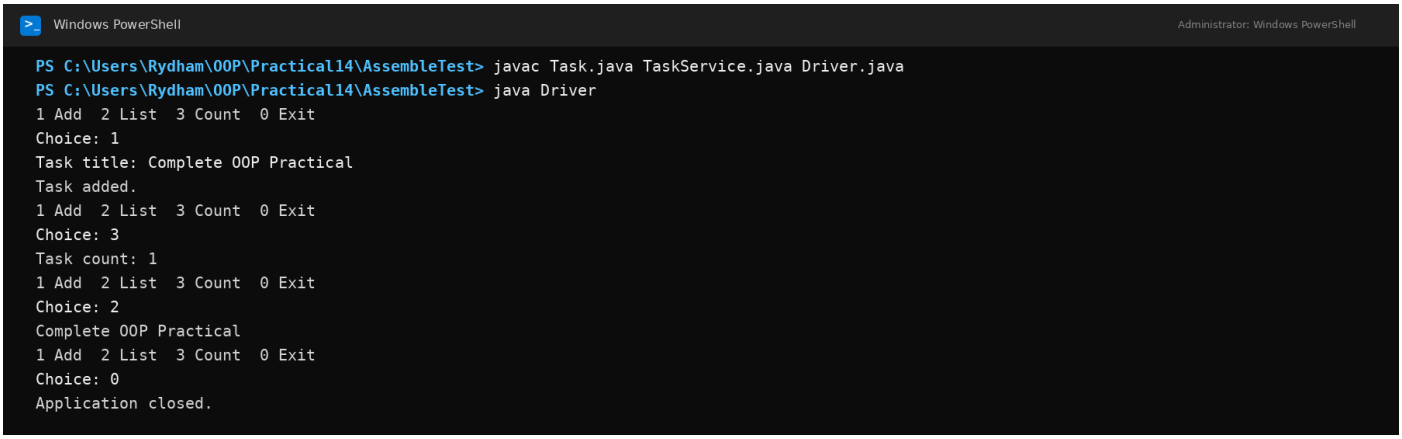
```java
import static org.junit.jupiter.api.Assertions.*;
import org.junit.jupiter.api.Test;

class TaskServiceTest {
    @Test
    void addingTaskIncreasesCount() {
        TaskService service = new TaskService();
        service.addTask("Prepare journal");
        assertEquals(1, service.getCount());
    }

    @Test
    void blankTaskThrowsException() {
        TaskService service = new TaskService();
        assertThrows(IllegalArgumentException.class,
                () -> service.addTask(" "));
    }

    @Test
    void taskTitleIsTrimmed() {
        TaskService service = new TaskService();
        service.addTask("  Java OOP  ");
        assertEquals("Java OOP", service.getTasks().get(0).getTitle());
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood layered software architecture (Service and Driver layers).
- Implemented automated assertion-based unit testing methodologies.
- Verified invariant enforcement and expected exception handling.
- Structured robust service validation rules.
- Gained end-to-end testing competency ahead of capstone project delivery.

PRACTICAL - 14

Aim: Modular packaging (For Advanced Learners)
(Task.java + TaskService.java + TaskStore.java + Driver.java + TaskStoreTest.java)

Code Screenshot:

Visual Studio Code - Task.java

Task.java

src > com > charusat > taskapp > model > Task.java

```java
package com.charusat.taskapp.model;
import java.io.Serializable;
import java.util.Objects;

public class Task implements Serializable {
    private static final long serialVersionUID = 1L;
    private final int id;
    private final String title;

    public Task(int id, String title) { this.id = id; this.title = title; }
    public int getId() { return id; }
    public String getTitle() { return title; }
    public String toString() { return id + " - " + title; }
    public boolean equals(Object obj) {
        if (!(obj instanceof Task other)) return false;
        return id == other.id && Objects.equals(title, other.title);
    }
    public int hashCode() { return Objects.hash(id, title); }
}
```

Visual Studio Code - TaskService.java

TaskService.java

src > com > charusat > taskapp > service > TaskService.java

```java
package com.charusat.taskapp.service;
import com.charusat.taskapp.model.Task;
import java.util.ArrayList;
import java.util.List;

public class TaskService {
    private final List<Task> tasks = new ArrayList<>();
    public void add(Task task) { tasks.add(task); }
    public List<Task> getAll() { return List.copyOf(tasks); }
    public void loadAll(List<Task> saved) {
        tasks.clear();
        tasks.addAll(saved);
    }
}
```

Visual Studio Code - TaskStore.java

TaskStore.java

src > com > charusat > taskapp > storage > TaskStore.java

```java
package com.charusat.taskapp.storage;
import com.charusat.taskapp.model.Task;
import java.io.*;
import java.nio.file.*;
import java.util.List;

public class TaskStore {
    public static void save(List<Task> tasks, Path file) throws IOException {
        try (ObjectOutputStream out = new ObjectOutputStream(
                Files.newOutputStream(file))) {
            out.writeObject(new java.util.ArrayList<>(tasks));
        }
    }

    @SuppressWarnings("unchecked")
    public static List<Task> load(Path file)
            throws IOException, ClassNotFoundException {
        try (ObjectInputStream in = new ObjectInputStream(
                Files.newInputStream(file))) {
            return (List<Task>) in.readObject();
        }
    }
}
```

Visual Studio Code - Driver.java

Driver.java

src > com > charusat > taskapp > app > Driver.java

```java
package com.charusat.taskapp.app;
import com.charusat.taskapp.model.Task;
import com.charusat.taskapp.service.TaskService;
import com.charusat.taskapp.storage.TaskStore;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import java.util.Scanner;

public class Driver {
    public static void main(String[] args) throws Exception {
        Path file = Path.of("tasks.dat");
        TaskService service = new TaskService();
        if (Files.exists(file)) {
            service.loadAll(TaskStore.load(file));
            System.out.println("Loaded tasks: " + service.getAll().size());
        }
        try (Scanner input = new Scanner(System.in)) {
            int choice;
            System.out.println("1 Add  2 List  0 Exit");
            do {
                System.out.print("Choice: ");
                choice = input.nextInt();
                input.nextLine();
                switch (choice) {
                    case 1 -> {
                        System.out.print("Task title: ");
                        service.add(new Task(service.getAll().size() + 1,
                                input.nextLine()));
                        System.out.println("Task added.");
                    }
                    case 2 -> service.getAll().forEach(System.out::println);
                    case 0 -> System.out.println("Closing application.");
                    default -> throw new IllegalArgumentException("Invalid action");
                }
            } while (choice != 0);
        }
        TaskStore.save(service.getAll(), file);
        List<Task> restored = TaskStore.load(file);
        System.out.println("Saved tasks: " + service.getAll().size());
        System.out.println("Reload verified: " + restored.equals(service.getAll()));
    }
}
```

Visual Studio Code - TaskStoreTest.java

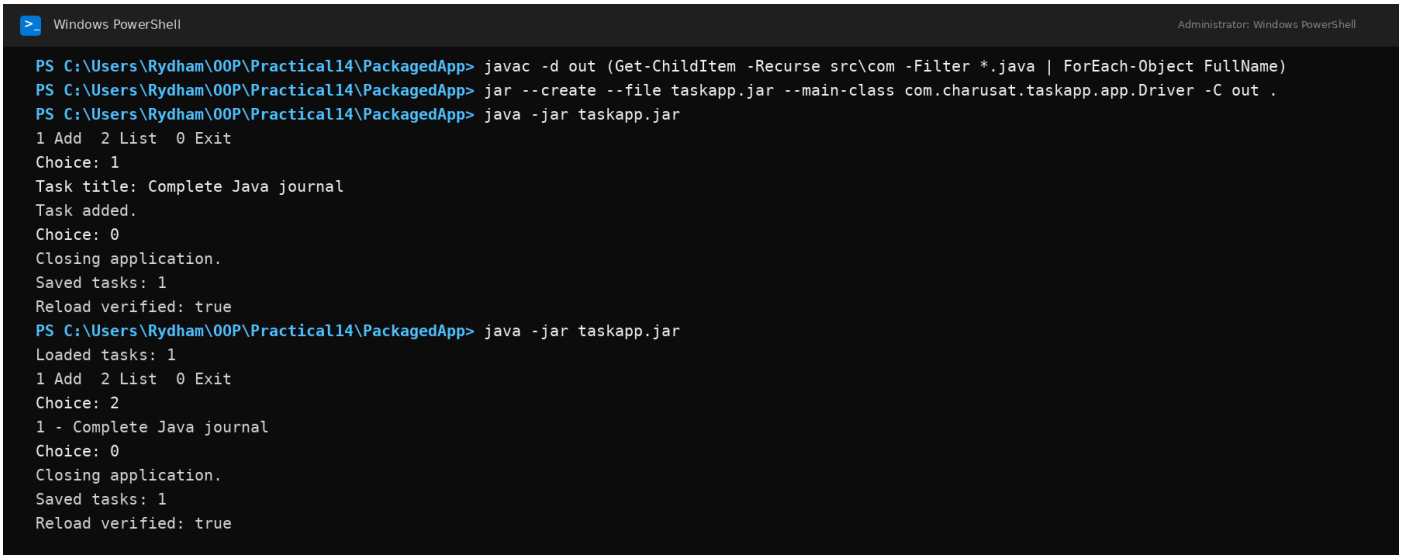
TaskStoreTest.java

src > test > java > com > charusat > taskapp > storage > TaskStoreTest.java

```java
package com.charusat.taskapp.storage;
import com.charusat.taskapp.model.Task;
import org.junit.jupiter.api.Test;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.List;
import static org.junit.jupiter.api.Assertions.assertEquals;

class TaskStoreTest {
    @Test
    void savedTasksReloadUnchanged() throws Exception {
        Path file = Files.createTempFile("tasks", ".dat");
        List<Task> original = List.of(
                new Task(1, "Study Java"), new Task(2, "Build JAR"));
        TaskStore.save(original, file);
        List<Task> loaded = TaskStore.load(file);
        assertEquals(original, loaded);
        Files.deleteIfExists(file);
    }
}
```

Output Screenshot:



Learning Outcome:

- Mastered Java package encapsulation and organizational structures.
- Learned how classes are bundled and deployed as runnable JAR archives.
- Verified modular integrity across package boundaries.
- Ensured application lifecycle continuity across deployment artifacts.
- Achieved industry-ready software packaging and delivery readiness.